

Univerzitet u Beogradu
Matematički fakultet

Klasterovanje fudbalera

Analiza, konstrukcija atributa, redukcija
dimenzionalnosti i primena algoritama
nenadgledanog učenja

Student: Mateja Janić

Predmet: Istraživanje Podataka 2

Beograd, 2026.

Sažetak

Cilj projekta bio je otkrivanje prirodnih grupa među profesionalnim fudbalerima primenom algoritama nenadgledanog mašinskog učenja. Za potrebe analize korišćeni su podaci o igračima, njihovim nastupima, minutaži, golovima, ulogama u timu, pozicijama, tržišnim vrednostima, klubovima, transferima i drugim karakteristikama.

Za potrebe projekta, korišćen je skup podataka *Football Data from Transfermarkt*, na linku [Skup podataka](#)

Početni podaci bili su raspoređeni u više međusobno povezanih tabela. Zbog toga je pre primene algoritama klasterovanja bilo neophodno izvršiti detaljno preprocesiranje, spajanje tabela, konstrukciju novih atributa, obradu nedostajućih vrednosti, enkodiranje kategorijskih promenljivih i standardizaciju numeričkih atributa.

Formirano je više verzija skupa podataka kako bi se ispitao uticaj pojedinih grupa atributa. Pored kompletnog skupa, analizirane su verzije bez atributa kluba, verzija sa ograničenim brojem država, kao i verzije dobijene redukcijom dimenzionalnosti pomoću metode glavnih komponenti.

Nad pripremljenim skupovima primenjeni su algoritmi K-sredina, GMM, hijerarhijsko klasterovanje, spektralno klasterovanje i DBSCAN. Rezultati su vrednovani pomoću silhouette koeficijenta, Davies–Bouldin koeficijenta, Calinski–Harabasz koeficijenta, broja dobijenih klastera, veličina klastera, broja instanci šuma i vremena izvršavanja.

Posebna pažnja posvećena je interpretaciji dobijenih grupa i njihovom prikazu u dve i tri dimenzije. Rezultati ukazuju na to da struktura klastera značajno zavisi od izbora atributa, načina redukcije dimenzionalnosti i pretpostavki samog algoritma.

Sadržaj

1	Uvod	6
2	Ciljevi projekta	6
3	Opis podataka	7
3.1	Izvorna struktura	7
3.2	Nivoi granularnosti	8
4	Pretprocesiranje podataka	8
4.1	Učitavanje podataka	8
4.2	Izbor relevantnih atributa	9
4.3	Referentni datum i izračunavanje starosti	10
4.4	Obrada nedostajućih vrednosti	10
4.5	Transferne naknade	11
4.6	Uklanjanje duplikata	11
4.7	Čuvanje pretprocesiranih podataka	12
5	Konstrukcija atributa	12
5.1	Atributi izvedeni iz nastupa	12
5.2	Uloga igrača u postavi	13
5.3	Pozicioni atributi	15
5.4	Atributi tržišne vrednosti	15
5.5	Spajanje tabela	17
5.6	Rezultat konstrukcije atributa	17
6	Priprema atributa i redukcija dimenzionalnosti	17
6.1	Razdvajanje numeričkih i kategoričkih atributa	18
6.2	One-hot enkodiranje	18
6.3	Problem atributa sa velikim brojem kategorija	19
6.4	Kompletan skup	19
6.5	Skup sa 30 najčešćih država	19
6.6	Skup bez atributa kluba	20
6.7	Standardizacija	20
6.8	Metoda glavnih komponenti	21
6.9	Izbor broja komponenti	21
7	Primena algoritama klasterovanja	23
7.1	Jedinstvena funkcija za pokretanje eksperimenata	23

8 K-sredina	25
8.1 Osnovna ideja	26
8.2 Izbor broja klastera	26
8.3 Prednosti i ograničenja	27
9 GMM	27
9.1 Osnovna ideja	27
9.2 Primena	28
9.3 Poređenje sa K-sredina	28
10 Hijerarhijsko klasterovanje	28
10.1 Osnovna ideja	29
10.2 Ward-ova metoda	29
10.3 Problem memorijske složenosti	29
10.4 Uzorak od 10 000 igrača	29
10.5 Primena algoritma	30
11 Spektralno klasterovanje	31
11.1 Osnovna ideja	31
11.2 Prednost algoritma	31
11.3 Računarska ograničenja	32
12 DBSCAN	32
12.1 Osnovna ideja	32
12.2 Prednosti	33
12.3 Ograničenja	33
12.4 Ispitane vrednosti parametara	33
12.5 Primeri dobijenih rezultata	34
13 Detaljniji eksperimenti	35
13.1 Klasterovanje nad različitim skupovima	35
13.2 Ograničavanje skupih algoritama	36
14 Evaluacija kvaliteta klasterovanja	36
14.1 Silhouette koeficijent	36
14.2 Davies–Bouldin indeks	37
14.3 Calinski–Harabasz indeks	37
14.4 Dodatni kriterijumi	38
14.5 Zašto jedna metrika nije dovoljna	38
15 Interpretacija klastera	38

15.1	VFM skup	44
15.1.1	Golmani	45
15.1.2	Srednji klaster	46
15.1.3	Afirmisani igrači	47
16	Vizualizacija rezultata	48
16.1	Ciljevi vizualizacije	48
16.2	Analiza doprinosa originalnih atributa atributima nastalim PCA metodom	49
16.2.1	X_no_countries_no_club_pca95	49
16.2.2	X_value_for_money	51
16.3	Dvodimenzionalna PCA projekcija	53
16.3.1	X_no_countries_no_club	54
16.3.2	X_value_for_money	56
16.4	Trodimenzionalna PCA projekcija	57
16.4.1	X_no_countries_no_club	59
16.4.2	X_value_for_money	61
17	Organizacija projekta po notebook-ovima	62
17.1	01_preprocessing.ipynb	62
17.2	02_feature_engineering.ipynb	63
17.3	03_feature_reduction.ipynb	63
17.4	04_clustering.ipynb	63
17.5	04_cluster_analysis.ipynb	64
17.6	06_more_specific_clustering.ipynb	64
17.7	07_cluster_visualization.ipynb	64
18	Čuvanje rezultata	64
19	Računarski izazovi	66
19.1	Veliki broj redova	66
19.2	Veliki broj atributa	66
19.3	Memorijska zahtevnost hijerarhijskog algoritma	66
19.4	Zahtevnost spektralnog klasterovanja	67
19.5	Dugo izvršavanje DBSCAN algoritma	67
19.6	Cena računanja silhouette koeficijenta	67
20	Diskusija	67
20.1	Uticaaj izbora atributa	67
20.2	Uticaaj PCA transformacije	68
20.3	Poređenje algoritama	68
20.4	Problem neuravnoteženih klastera	68

20.5 Interpretabilnost kao kriterijum	69
21 Ograničenja projekta	69
21.1 Kvalitet izvornih podataka	69
21.2 Tržišna vrednost nije čista mera kvaliteta	69
21.3 Različiti periodi karijere	70
21.4 PCA i gubitak interpretabilnosti	70
21.5 Uzorci kod skupih algoritama	70
22 Zaključak	70
23 Mogući pravci daljeg rada	71
A Pokretanje projekta	72
A.1 Kreiranje virtuelnog okruženja	72
A.2 Instalacija zavisnosti	72
A.3 Pokretanje Jupyter Notebook-a	72
B Korišćene biblioteke	73

1. Uvod

Savremena analiza fudbalskih podataka omogućava detaljno proučavanje karakteristika igrača koje prevazilaze klasične statistike, kao što su broj nastupa ili broj postignutih golova. Kombinovanjem podataka o minutaži, poziciji, ulozi u timu, tržišnoj vrednosti, transferima i istoriji nastupa moguće je formirati bogat opis svakog igrača.

Za razliku od klasifikacije, kod koje unapred postoje poznate ciljne klase, u ovom projektu nije postojala unapred definisana podela igrača. Zadatak je zato formulisan kao problem klasterovanja, odnosno pronalaženja grupa igrača koji su međusobno slični prema odabranim karakteristikama.

Glavni izazov nije bio samo izbor algoritma klasterovanja. Početni podaci bili su raspoređeni u više tabela različitih veličina i nivoa granularnosti. Jedan igrač se, na primer, mogao pojaviti jednom u tabeli igrača, više stotina puta u tabeli nastupa i više puta u tabeli tržišnih vrednosti. Pre klasterovanja bilo je potrebno sve informacije svesti na jedan red po igraču.

Rad je organizovan kroz sledeće Jupyter notebook-ove:

1. 01_preprocessing.ipynb
2. 02_feature_engineering.ipynb
3. 03_feature_reduction.ipynb
4. 04_clustering.ipynb
5. 05_cluster_analysis.ipynb
6. 06_more_specific_clustering.ipynb
7. 07_cluster_visualization.ipynb

Svaki notebook predstavlja jednu logičku fazu rada. Ovakva organizacija omogućava da se rezultati pojedinačnih faza sačuvaju i kasnije ponovo koriste bez ponovnog pokretanja vremenski zahtevnih algoritama.

2. Ciljevi projekta

Osnovni cilj projekta bio je pronalaženje smislenih grupa fudbalera pomoću metoda nenadgledanog učenja.

Detaljniji ciljevi bili su:

- objedinjavanje podataka iz više povezanih tabela;
- formiranje jednog vektora karakteristika za svakog igrača;
- konstrukcija atributa koji opisuju aktivnost igrača, poziciju, ulogu u timu i tržišnu

vrednost;

- obrada nedostajućih i ekstremnih vrednosti;
- poređenje različitih verzija skupa atributa;
- redukcija dimenzionalnosti primenom PCA metode;
- primena više algoritama klasterovanja;
- poređenje algoritama pomoću internih mera kvaliteta;
- analiza stabilnosti i interpretabilnosti klastera;
- vizuelizacija rezultata u dve i tri dimenzije.

Nije se očekivalo da svi algoritmi daju isti broj grupa niti identičnu podelu igrača. Različiti algoritmi imaju različite pretpostavke o obliku, gustini i raspodeli klastera, pa je jedan od ciljeva bio i ispitivanje uticaja tih pretpostavki na rezultate.

3. Opis podataka

3.1. Izvorna struktura

Početni skup bio je sastavljen od više CSV datoteka koje opisuju različite aspekte fudbalskih takmičenja i karijera igrača.

Najvažnije tabele bile su:

Tabela 1: Najvažnije izvorne tabele

Tabela	Opis
players.csv	Osnovni podaci o igračima, kao što su pozicija, visina, dominantna noga, državljanstvo, klub i tržišna vrednost.
appearances.csv	Podaci o nastupima igrača na utakmicama, minutaži, golovima, asistencijama i drugim događajima.
game_lineups.csv	Informacije o tome da li je igrač bio starter, rezerva ili kapiten i na kojoj je poziciji nastupao.
player_valuations.csv	Istorijske tržišne vrednosti igrača po datumima.
transfers.csv	Podaci o transferima i transfernim naknadama.
clubs.csv	Informacije o klubovima i domaćim takmičenjima.
games.csv	Osnovni podaci o odigranim utakmicama.
game_events.csv	Događaji sa utakmica, poput golova, izmena i kartona.

Ukupno je bilo dostupno približno 45 hiljada igrača. Pojedine tabele, posebno tabele

nastupa i postava, sadržale su više stotina hiljada ili više miliona redova.

Na primer:

- tabela igrača sadržala je približno 44 900 redova;
- tabela nastupa približno 1,86 miliona redova;
- tabela tržišnih vrednosti više od 600 hiljada redova;
- tabela postava više od 2,8 miliona redova;
- tabela događaja više od 1,2 miliona redova.

Zbog velike količine podataka bilo je važno izbegavati nepotrebna ponovna učitavanja i obrade.

3.2. Nivoi granularnosti

Tabele nisu imale isti nivo granularnosti:

- u tabeli igrača jedan red predstavlja jednog igrača;
- u tabeli nastupa jedan red predstavlja nastup igrača na određenoj utakmici;
- u tabeli postava jedan red predstavlja pojavljivanje igrača u sastavu za određenu utakmicu;
- u tabeli tržišnih vrednosti jedan red predstavlja procenu vrednosti igrača na određen datum.

Zbog toga nije bilo moguće direktno spojiti sve tabele i odmah primeniti algoritme klasterovanja. Najpre su tabele višeg nivoa granularnosti agregirane po identifikatoru igrača.

4. Pretprocesiranje podataka

Ova faza realizovana je u notebook-u `01_preprocessing.ipynb`.

4.1. Učitavanje podataka

Podaci su učitani pomoću biblioteke `pandas`. Nakon učitavanja provereni su:

- broj redova i kolona;
- nazivi atributa;
- tipovi podataka;
- broj nedostajućih vrednosti;
- broj duplikata;

- osnovne numeričke statistike;
- broj jedinstvenih kategorija.

Primer osnovnog učitavanja podataka:

```
1 import pandas as pd
2
3 players = pd.read_csv("../data/raw/players.csv")
4 appearances = pd.read_csv("../data/raw/appearances.csv")
5 lineups = pd.read_csv("../data/raw/game_lineups.csv")
6 valuations = pd.read_csv("../data/raw/player_valuations.csv")
7 transfers = pd.read_csv("../data/raw/transfers.csv")
8 clubs = pd.read_csv("../data/raw/clubs.csv")
```

Listing 1: Učitavanje osnovnih tabela

4.2. Izbor relevantnih atributa

Iz tabele igrača zadržani su atributi koji mogu doprineti razlikovanju profila igrača.

Među osnovnim atributima nalazili su se:

- identifikator igrača;
- starost;
- visina;
- broj nastupa za reprezentaciju;
- broj golova za reprezentaciju;
- trenutna tržišna vrednost;
- najveća istorijska tržišna vrednost;
- glavna pozicija;
- uža pozicija;
- dominantna noga;
- država rođenja;
- državljanstvo;
- trenutni klub;
- domaće takmičenje trenutnog kluba.

Atributi koji su predstavljali samo tekstualne identifikatore, URL adrese, imena i druge informacije bez direktnog analitičkog značaja nisu uključeni u ulaz za klasterovanje.

4.3. Referentni datum i izračunavanje starosti

Starost igrača određena je u odnosu na jedinstven referentni datum. Na taj način izbegnuto je da vreme pokretanja notebook-a utiče na rezultat.

Korišćen je referentni datum:

$$d_{\text{ref}} = 12.06.2026.$$

Starost je računata kao približna razlika između referentnog datuma i datuma rođenja:

$$\text{starost} = \frac{d_{\text{ref}} - d_{\text{rođenja}}}{365.25}.$$

```

1 reference_date = pd.Timestamp("2026-06-12")
2
3 players["date_of_birth"] = pd.to_datetime(
4     players["date_of_birth"],
5     errors="coerce"
6 )
7
8 players["age"] = (
9     reference_date - players["date_of_birth"]
10 ).dt.days / 365.25

```

Listing 2: Izračunavanje starosti igrača

Za mali broj igrača datum rođenja nije bio poznat, pa je i izračunata starost ostala nedostajuća.

4.4. Obrada nedostajućih vrednosti

Nedostajuće vrednosti nisu obrađene na isti način za sve attribute.

Za numeričke attribute korišćena je medijana:

$$x_i = \begin{cases} x_i, & \text{ako je vrednost poznata,} \\ \text{median}(X), & \text{ako vrednost nedostaje.} \end{cases}$$

Medijana je izabrana umesto srednje vrednosti zato što je robusnija na ekstremne vrednosti. Ovo je naročito važno za tržišne vrednosti, minutažu i broj nastupa, čije su raspodele izrazito asimetrične.

Za kategorijske attribute nedostajuće vrednosti sačuvane su kao posebna kategorija ili

su tokom one-hot enkodiranja predstavljene posebnim indikatorskim atributom.

```
1 numeric_columns = [  
2     "age",  
3     "height_in_cm",  
4     "international_caps",  
5     "international_goals",  
6     "market_value_in_eur",  
7     "highest_market_value_in_eur"  
8 ]  
9  
10 for column in numeric_columns:  
11     players[column] = players[column].fillna(  
12         players[column].median()  
13     )
```

Listing 3: Primer imputacije numeričkih atributa

4.5. Transferne naknade

Kod podataka o transferima, nedostajuća transferna naknada nije nužno značila grešku u podacima. U velikom broju slučajeva mogla je označavati slobodan transfer ili transfer bez javno poznate naknade.

U okviru izabrane obrade nedostajuća transferna naknada zamenjena je nulom:

```
1 transfers["transfer_fee"] = (  
2     transfers["transfer_fee"].fillna(0)  
3 )
```

Listing 4: Obrada nedostajućih transfernih naknada

Ova odluka mora se imati u vidu prilikom interpretacije, jer vrednost nula može objediniti stvarno besplatne transfere i transfere sa nepoznatom naknadom.

4.6. Uklanjanje duplikata

Duplikati su proveravani u odnosu na prirodni ključ odgovarajuće tabele. Kod tabele igrača očekivan je jedan red po `player_id`, dok je kod drugih tabela kombinacija identifikatora igrača, utakmice i datuma mogla predstavljati složeniji ključ.

Uklanjanje su samo redovi za koje je utvrđeno da predstavljaju stvarna ponavljanja, kako se ne bi slučajno uklonili legitimni višestruki nastupi istog igrača.

4.7. Čuvanje pretprocesiranih podataka

Rezultati pretprocesiranja sačuvani su u direktorijumu za obrađene podatke. Time je omogućeno da naredni notebook-ovi koriste gotove tabele bez ponavljanja početne obrade.

Primer:

```
1 players.to_csv(  
2     "../data/processed/players_preprocessed.csv",  
3     index=False  
4 )
```

Listing 5: Čuvanje obrađenih podataka

5. Konstrukcija atributa

Konstrukcija novih atributa izvršena je u notebook-u `02_feature_engineering.ipynb`.

Cilj ove faze bio je da se podaci iz tabela nastupa, postava, pozicija, tržišnih vrednosti i transfera agregiraju na nivou igrača.

5.1. Atributi izvedeni iz nastupa

Za svakog igrača izračunati su agregati na osnovu svih njegovih nastupa.

Najvažniji atributi bili su:

- broj odigranih utakmica;
- ukupna minutaža;
- prosečna minutaža po nastupu;
- maksimalna minutaža na jednoj utakmici;
- standardna devijacija minutaže;
- ukupan broj golova;
- prosečan broj golova po nastupu;
- broj žutih i crvenih kartona, kada su bili dostupni;
- druge statistike nastupa prisutne u izvornoj tabeli.

Broj nastupa igrača i definisan je kao:

$$M_i = \sum_{j=1}^{n_i} 1,$$

dok je ukupna minutaža:

$$T_i = \sum_{j=1}^{n_i} m_{ij},$$

gde je m_{ij} minutaža igrača i na utakmici j .

Prosečna minutaža računata je kao:

$$\overline{m}_i = \frac{1}{M_i} \sum_{j=1}^{M_i} m_{ij}.$$

```
1 appearance_features = (  
2     appearances  
3     .groupby("player_id")  
4     .agg(  
5         matches_played=("game_id", "nunique"),  
6         total_minutes=("minutes_played", "sum"),  
7         average_minutes=("minutes_played", "mean"),  
8         maximum_minutes=("minutes_played", "max"),  
9         minutes_std=("minutes_played", "std"),  
10        total_goals=("goals", "sum")  
11    )  
12    .reset_index()  
13 )
```

Listing 6: Agregiranje podataka o nastupima

Standardna devijacija minutaže korišćena je kao mera stabilnosti uloge igrača. Mala vrednost može ukazivati na stabilnu minutažu, dok velika vrednost može ukazivati na česte promene između pune utakmice, ulaska sa klupe i neigranja.

5.2. Uloga igrača u postavi

Iz tabele postava izvedeni su atributi koji opisuju koliko često je igrač bio:

- starter;
- rezerva;
- kapiten.

Za svaku ulogu računati su i apsolutni brojevi i relativni udeli.

Na primer, udeo utakmica u kojima je igrač bio starter definisan je kao:

$$p_{\text{starter},i} = \frac{\text{broj nastupa u početnoj postavi}}{\text{ukupan broj pojavljivanja u postavi}}.$$

Relativni atributi su korisni zato što omogućavaju poređenje igrača sa različitim ukupnim brojem utakmica.

```
1 lineups["is_starter"] = (  
2     lineups["type"].eq("starting_lineup").astype(int)  
3 )  
4  
5 lineups["is_substitute"] = (  
6     lineups["type"].eq("substitutes").astype(int)  
7 )  
8  
9 lineups["is_captain"] = (  
10    lineups["team_captain"].fillna(0).astype(int)  
11 )  
12  
13 role_features = (  
14     lineups  
15     .groupby("player_id")  
16     .agg(  
17         starter_count=("is_starter", "sum"),  
18         substitute_count=("is_substitute", "sum"),  
19         captain_count=("is_captain", "sum"),  
20         lineup_count=("game_id", "nunique")  
21     )  
22     .reset_index()  
23 )
```

Listing 7: Konstrukcija atributa uloge

Nakon agregiranja formirani su procentualni atributi:

```
1 role_features["starter_percentage"] = (  
2     role_features["starter_count"]  
3     / role_features["lineup_count"].replace(0, 1)  
4 )  
5  
6 role_features["substitute_percentage"] = (  
7     role_features["substitute_count"]  
8     / role_features["lineup_count"].replace(0, 1)  
9 )  
10
```

```
11 role_features["captain_percentage"] = (  
12     role_features["captain_count"]  
13     / role_features["lineup_count"].replace(0, 1)  
14 )
```

Listing 8: Relativni atributi uloga

5.3. Pozicioni atributi

Pored osnovne pozicije iz tabele igrača, korišćeni su podaci o stvarnim pozicijama na kojima je igrač nastupao.

Za svaku posmatranu poziciju formirani su:

- broj nastupa na toj poziciji;
- procenat nastupa na toj poziciji.

Na ovaj način bilo je moguće razlikovati:

- specijalizovane igrače koji gotovo uvek igraju na jednoj poziciji;
- univerzalne igrače koji nastupaju na više pozicija;
- igrače čija formalna pozicija ne odgovara potpuno njihovoj stvarnoj ulozi.

Ako je c_{ik} broj nastupa igrača i na poziciji k , a N_i ukupan broj nastupa za koje je pozicija poznata, tada je:

$$p_{ik} = \frac{c_{ik}}{N_i}.$$

U skupu je formirano približno 17 pozicionih kategorija, zajedno sa njihovim apsolutnim i relativnim atributima.

5.4. Atributi tržišne vrednosti

Istorija tržišnih vrednosti agregirana je za svakog igrača.

Formirani su atributi:

- broj dostupnih procena vrednosti;
- prosečna tržišna vrednost;
- minimalna vrednost;
- maksimalna vrednost;
- standardna devijacija vrednosti;
- prva poznata vrednost;

- poslednja poznata vrednost;
- apsolutna promena vrednosti;
- procentualna promena vrednosti.

Apsolutna promena vrednosti definisana je kao:

$$\Delta V_i = V_{i,\text{poslednja}} - V_{i,\text{prva}}.$$

Procentualna promena računata je kao:

$$\Delta V_i^{(\%)} = \frac{V_{i,\text{poslednja}} - V_{i,\text{prva}}}{V_{i,\text{prva}}} \cdot 100.$$

Kada je početna vrednost bila jednaka nuli ili nije bila poznata, procentualna promena nije računata direktnom formulom zbog deljenja nulom.

```
1 valuations = valuations.sort_values(  
2     ["player_id", "date"]  
3 )  
4  
5 valuation_features = (  
6     valuations  
7     .groupby("player_id")  
8     .agg(  
9         valuation_count=("market_value_in_eur", "count"),  
10        valuation_mean=("market_value_in_eur", "mean"),  
11        valuation_min=("market_value_in_eur", "min"),  
12        valuation_max=("market_value_in_eur", "max"),  
13        valuation_std=("market_value_in_eur", "std"),  
14        first_valuation=("market_value_in_eur", "first"),  
15        latest_valuation=("market_value_in_eur", "last")  
16    )  
17    .reset_index()  
18 )  
19  
20 valuation_features["valuation_growth"] = (  
21     valuation_features["latest_valuation"]  
22     - valuation_features["first_valuation"]  
23 )
```

Listing 9: Agregiranje istorijskih tržišnih vrednosti

5.5. Spajanje tabela

Sve agregirane tabele spojene su sa osnovnom tabelom igrača pomoću atributa `player_id`.

Korišćeno je levo spajanje kako bi se zadržali svi igrači iz osnovne tabele, uključujući one koji nisu imali podatke u nekoj od pomoćnih tabela.

```
1 players_features = (  
2     players  
3     .merge(  
4         appearance_features,  
5         on="player_id",  
6         how="left"  
7     )  
8     .merge(  
9         role_features,  
10        on="player_id",  
11        how="left"  
12    )  
13    .merge(  
14        valuation_features,  
15        on="player_id",  
16        how="left"  
17    )  
18 )
```

Listing 10: Spajanje agregiranih atributa

Nedostajuće vrednosti nastale nakon spajanja u velikom broju slučajeva značile su da igrač nema zabeležen nastup, ulogu ili procenu vrednosti. Kod numeričkih atributa takve vrednosti zamenjene su nulom.

5.6. Rezultat konstrukcije atributa

Nakon spajanja i konstrukcije izvedenih atributa dobijen je skup u kojem jedan red predstavlja jednog igrača.

Broj atributa rastao je postepeno. Nakon dodavanja pozicionih, statističkih i vrednosnih karakteristika skup je sadržao nešto više od 100 osnovnih i izvedenih kolona, pre one-hot enkodiranja kategorijskih atributa.

6. Priprema atributa i redukcija dimenzionalnosti

Ova faza realizovana je u notebook-u `03_feature_reduction.ipynb`.

6.1. Razdvajanje numeričkih i kategoričkih atributa

Atributi su podeljeni na dve osnovne grupe:

- numeričke attribute;
- kategoričke attribute.

Numerički atributi mogli su se direktno standardizovati, dok je kategoričke attribute bilo potrebno numerički predstaviti.

6.2. One-hot enkodiranje

Za kategoričke attribute korišćeno je one-hot enkodiranje.

Ako atribut ima kategorije $c_1, c_2, \dots, c_k$, formira se k binarnih atributa:

$$x_j = \begin{cases} 1, & \text{ako instanca pripada kategoriji } c_j, \\ 0, & \text{inače.} \end{cases}$$

Enkodirani su atributi poput:

- glavne pozicije;
- uže pozicije;
- dominantne noge;
- države rođenja;
- državljanstva;
- identifikatora domaćeg takmičenja;
- određenih atributa kluba.

```
1 categorical_columns = [  
2     "position",  
3     "sub_position",  
4     "foot",  
5     "country_of_birth",  
6     "country_of_citizenship",  
7     "current_club_domestic_competition_id"  
8 ]  
9  
10 encoded_data = pd.get_dummies(  
11     players_features,  
12     columns=categorical_columns,  
13     dummy_na=True
```

14)

Listing 11: One-hot enkodiranje

Kompletni enkodirani skup sadržao je približno 364 atributa.

6.3. Problem atributa sa velikim brojem kategorija

Države rođenja i državljanstva imale su približno 195 različitih vrednosti. One-hot enkodiranje takvih atributa značajno povećava broj dimenzija.

Sličan problem javlja se kod atributa kluba. Veliki broj klubova može dovesti do toga da algoritam grupiše igrače prema klupskoj pripadnosti, umesto prema njihovom sportskom profilu.

Zbog toga je formirano više verzija skupa.

6.4. Kompletni skup

Kompletni skup, označen kao `X_full`, sadržao je sve izabrane numeričke i enkodirane kategoričke attribute.

Prednost ovog skupa je najveća količina dostupnih informacija.

Nedostaci su:

- veliki broj dimenzija;
- veća računarska zahtevnost;
- mogućnost da retke kategorije imaju preveliki uticaj;
- mogućnost da klupski atributi dominiraju nad igračkim karakteristikama.

Napomena: Iz narednih skupova je takođe izbačena informacija o državi rođenja. U velikoj većini slučajeva, država rođenja nije veoma bitna karakteristika, a značajno utiče na dimenzionalnost skupa podataka.

6.5. Skup sa 30 najčešćih država

Formirana je verzija `X_top30`.

Kod atributa države za koju igrač nastupa zadržano je 30 najzastupljenijih država, dok su sve ostale objedinjene u kategoriju `OTHER`.

```
1 top_30_countries = (  
2     df_top30["country_of_citizenship"]  
3     .value_counts()  
4     .head(30)
```

```
5     .index
6 )
7
8 df_top30["country_of_citizenship"] = np.where(
9     df_top30["country_of_citizenship"].isin(top_30_countries),
10    df_top30["country_of_citizenship"],
11    "OTHER"
12 )
```

Listing 12: Grupisanje retkih država

Ovaj pristup smanjuje broj dimenzija, ali i dalje čuva informacije o najzastupljenijim državama.

Dobijeni skup sadržao je približno 183 atributa.

6.6. Skup bez atributa kluba

Formirana je i verzija `X_player_only`, iz koje su uklonjeni atributi koji direktno identifikuju klub ili snažno zavise od kluba.

Cilj je bio da se igrači grupišu prvenstveno prema sopstvenim karakteristikama:

- poziciji;
- nastupima;
- minutaži;
- ulozi;
- tržišnoj vrednosti;
- istoriji vrednosti;
- reprezentativnom iskustvu.

Ova verzija sadržala je približno 177 atributa.

6.7. Standardizacija

Standardizacija je izvršena pomoću transformacije:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

gde su μ_j i σ_j srednja vrednost i standardna devijacija atributa j .

```
1 from sklearn.preprocessing import StandardScaler
2
```

```
3 scaler = StandardScaler()  
4  
5 X_scaled = scaler.fit_transform(  
6     encoded_data.drop(columns=["player_id"])  
7 )
```

Listing 13: Standardizacija atributa

Standardizacija je neophodna jer se algoritmi zasnovani na rastojanju mogu ponašati loše kada atributi imaju različite skale.

Na primer, tržišna vrednost izražena u evrima mogla bi potpuno dominirati nad binarnim pozicionim atributima kada standardizacija ne bi bila primenjena.

6.8. Metoda glavnih komponenti

PCA, odnosno metoda glavnih komponenti, korišćena je za redukciju dimenzionalnosti.

Neka je standardizovani skup podataka predstavljen matricom:

$$X \in \mathbb{R}^{n \times p}.$$

PCA pronalazi ortogonalne pravce $w_1, w_2, \dots, w_k$ koji redom maksimizuju varijansu projekcija podataka.

Prva komponenta rešava problem:

$$w_1 = \arg \max_{\|w\|=1} \text{Var}(Xw).$$

Naredne komponente maksimizuju preostalu varijansu uz uslov ortogonalnosti u odnosu na prethodne komponente.

6.9. Izbor broja komponenti

Broj komponenti nije biran proizvoljno, već prema željenom procentu objašnjene varijanse.

Ako je λ_j sopstvena vrednost koja odgovara komponenti j , udeo objašnjene varijanse je:

$$r_j = \frac{\lambda_j}{\sum_{\ell=1}^p \lambda_{\ell}}.$$

Broj komponenti k određen je uslovom:

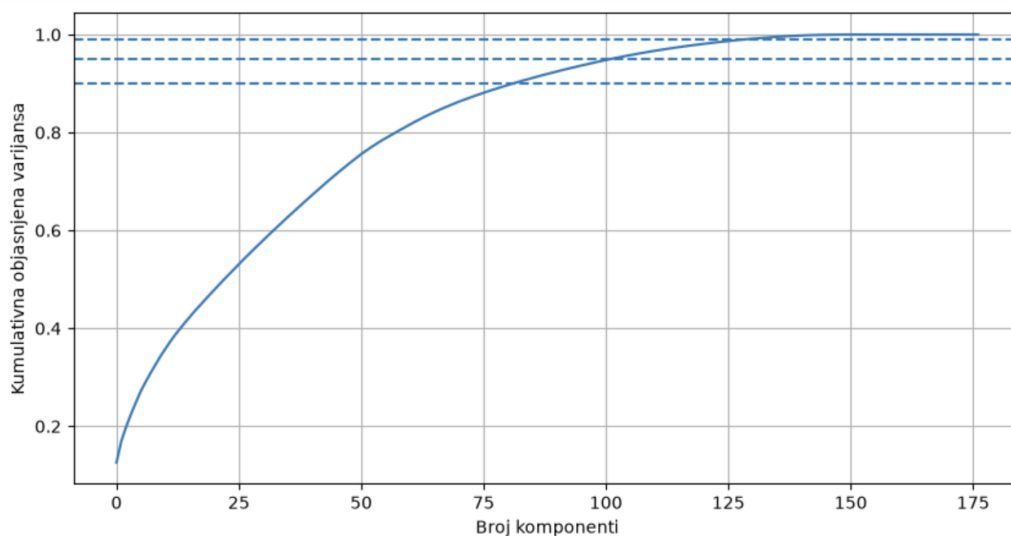
$$\sum_{j=1}^k r_j \geq \tau,$$

gde je τ izabrani prag, na primer 0,90 ili 0,95.

Nad poslednjim spomenutim skupom podataka, koji je sadržao samo najučestalijih 30 država i nije sadržao informacije o klupskim atributima, ispitivano je koji procenat zadržane varijanse će napraviti dovoljno veliku redukciju dimenzionalnosti podataka. Prva analiza je nastojala da zadrži 95% varijanse, ali je time napravljena redukcija na 107 atributa, što iako jeste značajno, nije smatrano za dovoljno. Iz tog razloga, odlučeno je da se pokuša sa zadržavanjem 90% varijanse. U tom slučaju, dimenzionalnost je spuštена na 82 atributa, što je za ovaj skup podataka i uzeto kao dovoljno.

```
1 from sklearn.decomposition import PCA
2 from matplotlib import pyplot as plt
3
4 pca = PCA()
5 pca.fit(X_player_only)
6
7 cum_var = np.cumsum(pca.explained_variance_ratio_)
8
9 plt.figure(figsize=(10, 5))
10 plt.plot(cum_var)
11 plt.axhline(y=0.90, linestyle="--")
12 plt.axhline(y=0.95, linestyle="--")
13 plt.axhline(y=0.99, linestyle="--")
14 plt.xlabel("Broj komponenti")
15 plt.ylabel("Kumulativna objasnjena varijansa")
16 plt.grid(True)
17 plt.show()
```

Listing 14: Grafik zavisnosti objašnjenje/zadržane varijanse od broja atributa



Slika 1: Rezultat pokretanja prethodnog koda.

U narednim fazama, kada su određeni najbolji načini klasterovanja igrača, detaljnije su analizirani PCA atributi, odnosno uticaj originalnih atributa na one dobijene PCA metodom. O tome će više reči biti u narednim poglavljima.

7. Primena algoritama klasterovanja

Osnovni eksperimenti realizovani su u notebook-u `04_clustering.ipynb`.

Detaljniji i ciljani eksperimenti realizovani su u notebook-u `05_more_specific_clustering.ipynb`.

Primenjeno je pet algoritama:

1. K-sredina;
2. GMM;
3. hijerarhijsko klasterovanje;
4. spektralno klasterovanje;
5. DBSCAN.

7.1. Jedinstvena funkcija za pokretanje eksperimenata

Kako bi se rezultati različitih algoritama zapisivali u istom formatu, napravljen3 su pomoćne funkcije za:

- pokretanje algoritma;
- računanje silhouette koeficijenta;
- čuvanje rezultata u tabeli.

U narednom kodu su prikazane sve tri navedene funkcije:

```
1 def evaluate_clustering(X, labels):
2     labels = np.asarray(labels)
3
4     n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
5     n_noise = np.sum(labels == -1)
6
7     if n_clusters < 2:
8         return {
9             "n_clusters": n_clusters,
10            "n_noise": n_noise,
11            "silhouette": np.nan,
12            "davies_bouldin": np.nan,
13            "calinski_harabasz": np.nan
14        }
15
16     mask = labels != -1
17
18     return {
19         "n_clusters": n_clusters,
20         "n_noise": n_noise,
21         "silhouette": silhouette_score(X[mask], labels[mask]),
22         "davies_bouldin": davies_bouldin_score(X[mask], labels[
23             mask]),
24         "calinski_harabasz": calinski_harabasz_score(X[mask],
25             labels[mask])
26     }
27
28 def save_result(dataset_name, algorithm_name, labels, metrics):
29     cluster_df = pd.DataFrame({
30         "player_id": player_ids["player_id"],
31         "cluster": labels
32     })
33
34     cluster_path = RESULTS_DIR / f"{dataset_name}_{algorithm_name}
35         _clusters.csv"
36     cluster_df.to_csv(cluster_path, index=False)
37
38     metrics_df = pd.DataFrame([
39         "dataset": dataset_name,
40         "algorithm": algorithm_name,
```

```
38     **metrics
39     })
40
41     metrics_path = RESULTS_DIR / f"{dataset_name}_{algorithm_name}
42     _metrics.csv"
43     metrics_df.to_csv(metrics_path, index=False)
44
45     return metrics_df
46
47 def run_algorithm(X, dataset_name, algorithm_name, model):
48     print(f"Running {algorithm_name} on {dataset_name}...")
49     start = time.time()
50
51     labels = model.fit_predict(X)
52
53     elapsed = time.time() - start
54     metrics = evaluate_clustering(X, labels)
55     metrics["time_seconds"] = elapsed
56
57     result = save_result(dataset_name, algorithm_name, labels,
58                          metrics)
59
60     print(f"Finished {algorithm_name} on {dataset_name} in {
61           elapsed:.2f}s")
62     display(result)
63
64     return labels, result
```

Listing 15: Prikaz funkcija napravljenih za usklađeno pokretanje algoritama klasterovanja

Napomena: Oznaka `RESULTS_DIR` u prikazanom kodu predstavlja promenljivu koja je definisana pre prikazanih funkcija i označava relativnu putanju do direktorijuma u kojem su čuvani rezultati klasterovanja.

Kod DBSCAN algoritma metrike su računate nad tačkama koje nisu označene kao šum.

8. K-sredina

8.1. Osnovna ideja

K-sredina deli podatke na unapred zadat broj klastera K .

Cilj algoritma je minimizacija funkcije:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2,$$

gde je C_k skup instanci u klasteru k , a μ_k centroid tog klastera.

Algoritam iterativno:

1. dodeljuje svaku instancu najbližem centroidu;
2. ponovo računa centroide kao srednje vrednosti dodeljenih instanci;
3. ponavlja postupak do konvergencije.

8.2. Izbor broja klastera

K-sredina zahteva da broj klastera bude zadat unapred.

Razmatrane su vrednosti:

$$K \in \{2, 3, 4, 5, 6, 7, 10\}.$$

Međutim, treba imati na umu da se za $k = 2$ u gotovo svim slučajevima klastera dobijaju klasteri koji nemaju mnogo smisla, odnosno veoma su neuravnoteženi. Iz tog razloga, iako je po metrikama $k = 2$ bilo često najbolje rešenje, ono nije uzimano u obzir za dalju analizu.

```
1 from sklearn.cluster import KMeans
2
3 for k in [3, 4, 5, 6, 7, 10]:
4     labels, res = run_algorithm(
5         X_full,
6         "full",
7         "kmeans",
8         KMeans(n_clusters=k, random_state=42, n_init=10)
9     )
10    full_results.append(res)
```

Listing 16: Primer pokretanja K-sredina za više vrednosti K nad celim skupom podataka

8.3. Prednosti i ograničenja

Prednosti algoritma K-sredina u ovom projektu bile su:

- relativno brzo izvršavanje;
- mogućnost rada nad celim skupom;
- jednostavna interpretacija preko centroida;
- lako poređenje različitih vrednosti K .

Ograničenja su:

- broj klastera mora biti poznat unapred;
- algoritam favorizuje približno sferne klastere;
- osetljiv je na skaliranje atributa;
- osetljiv je na ekstremne vrednosti;
- različite inicijalizacije mogu dati različita rešenja.

Korišćena je inicijalizacija `k-means++` i više ponovljenih inicijalizacija kako bi se smanjio uticaj slučajnog početnog izbora centroida.

9. GMM

9.1. Osnovna ideja

Model GMM (eng. Gaussian Mixture Model) pretpostavlja da su podaci generisani kao kombinacija više višedimenzionih normalnih raspodela.

Gustina podataka modeluje se kao:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

gde su:

- π_k težina komponente;
- μ_k vektor srednjih vrednosti;
- Σ_k matrica kovarijanse;
- K broj komponenti.

Za razliku od K-sredina, GMM prirodno omogućava meku pripadnost klasterima. Za svaku instancu dobija se verovatnoća pripadnosti svakoj komponenti.

9.2. Primena

Testirano je više vrednosti broja komponenti, usklađenih sa vrednostima korišćenim kod K-sredina.

```
1 from sklearn.mixture import GaussianMixture
2
3 top30_gmm_results = []
4
5 for n in [2, 3, 4, 5, 6, 7, 10]:
6     labels, res = run_algorithm(
7         X_top30,
8         "top30",
9         f"gmm_{n}",
10        GaussianMixture(
11            n_components=n,
12            random_state=42,
13            covariance_type="full"
14        )
15    )
16    top30_gmm_results.append(res)
```

Listing 17: Primer pokretanja GMM klasterovanja nad skupom podataka koji sadrži samo 30 najučestalijih zemalja

Dijagonalna matrica kovarijanse može značajno smanjiti računarsku zahtevnost u prostoru sa velikim brojem dimenzija. Potpuna matrica kovarijanse bila bi znatno zahtevnija i mogla bi biti numerički nestabilna.

9.3. Poređenje sa K-sredina

K-sredina svakom igraču dodeljuje tačno jedan klaster, dok GMM omogućava da igrač ima, na primer:

$$P(C_1 | x) = 0.55, \quad P(C_2 | x) = 0.40, \quad P(C_3 | x) = 0.05.$$

Ovakav rezultat može biti koristan kod igrača koji imaju mešoviti profil, na primer kod univerzalnih veznih igrača ili fudbalera koji pokrivaju više pozicija.

10. Hijerarhijsko klasterovanje

10.1. Osnovna ideja

Korišćen je aglomerativni pristup. Na početku svaki igrač predstavlja poseban klaster, a zatim se najbliži klasteri postepeno spajaju.

Postupak se ponavlja dok se ne dobije željeni broj klastera.

10.2. Ward-ova metoda

Korišćena je Ward-ova metoda povezivanja, koja u svakom koraku bira spajanje koje najmanje povećava ukupnu varijansu unutar klastera.

Povećanje kriterijuma pri spajanju klastera A i B može se zapisati kao:

$$\Delta(A, B) = \frac{|A||B|}{|A| + |B|} \|\mu_A - \mu_B\|_2^2.$$

Ward-ova metoda je pogodna za standardizovane numeričke podatke i često daje relativno kompaktne klastere.

10.3. Problem memorijske složenosti

Hijerarhijsko klasterovanje zahteva računanje i čuvanje velikog broja međusobnih rastojanja.

Za n instanci broj parova je:

$$\frac{n(n-1)}{2}.$$

Za približno 45 hiljada igrača to je oko milijardu parova. Zbog toga je pokretanje nad celim skupom dovelo do prevelike potrošnje memorije i prekida rada kernela.

10.4. Uzorak od 10 000 igrača

Hijerarhijsko klasterovanje zato je ograničeno na uzorak od 10 000 instanci.

Uzorak nije biran potpuno nasumično, već stratifikovano prema glavnoj poziciji igrača. Time je očuvana približna raspodela golmana, odbrambenih, veznih i napadačkih igrača.

```
1 from sklearn.model_selection import train_test_split
2
3 sample_size = 10000
4
5 sample_meta, _ = train_test_split(
6     df_meta,
```

```
7     train_size=sample_size,  
8     random_state=42,  
9     stratify=df_meta["position"]  
10 )  
11  
12 sample_indices = sample_meta.index.to_numpy()  
13  
14 X_top30_np = pd.read_csv(DATASET_DIR / "X_top30.csv").values  
15  
16 X_top30_sample = X_top30_np[sample_indices]  
17 player_ids_sample = player_ids.iloc[sample_indices].reset_index(  
    drop=True)
```

Listing 18: Stratifikovani uzorak od 10 000 igrača

Možemo primetiti da je pored uzimanja uzorka iz skupa podataka takođe bitno i zapamtiti identifikatore igrača koji su izabrani.

10.5. Primena algoritma

```
1 from sklearn.cluster import AgglomerativeClustering  
2  
3 player_ids_original = player_ids.copy()  
4 player_ids = player_ids_sample.copy()  
5  
6 top30_hierarchical_results = []  
7  
8 for k in [2, 3, 4, 5, 6, 7, 10]:  
9     labels, res = run_algorithm(  
10         X_top30_sample,  
11         "top30_sample10000",  
12         f"hierarchical_{k}",  
13         AgglomerativeClustering(  
14             n_clusters=k,  
15             linkage="ward"  
16         )  
17     )  
18     top30_hierarchical_results.append(res)  
19  
20 player_ids = player_ids_original.copy()
```

Listing 19: Aglomerativno klasterovanje

Algoritam hijerarhijskog klasterovanja je pokretan na isti način kao i ostali algoritmi, sa time što je bilo potrebno prilagoditi ga činjenici da se sada algoritam pokreće na uzorku od 10 000 instanci. Iz tog razloga je bilo potrebno zapamtiti originalne identifikatore igrača, zameniti ih onima iz uzorka, pokrenuti algoritam, a zatim, na kraju, vratiti identifikatore na početne. Potreba za ovakvim načinom se javila iz definicije funkcije za pokretanje algoritama.

11. Spektralno klasterovanje

11.1. Osnovna ideja

Spektralno klasterovanje posmatra podatke kao graf.

Svaka instanca predstavlja čvor, dok težina grane između dve instance opisuje njihovu sličnost.

Matrica sličnosti može se definisati RBF jezgrom:

$$W_{ij} = \exp \left(-\gamma \|x_i - x_j\|_2^2 \right).$$

Na osnovu matrice sličnosti formira se matrica stepena:

$$D_{ii} = \sum_j W_{ij},$$

i Laplasijan grafa:

$$L = D - W.$$

Klasterovanje se zatim vrši u prostoru sopstvenih vektora Laplasijana.

11.2. Prednost algoritma

Za razliku od K-sredina, spektralno klasterovanje može pronaći klastere složenijih i nelinearnih oblika.

Ovo može biti značajno kada grupe igrača nisu razdvojene linearnim ili sfernim granicama.

11.3. Računarska ograničenja

Glavni problem je konstrukcija matrice sličnosti. Za n instanci puna matrica ima n^2 elemenata.

Za približno 45 hiljada igrača to bi značilo više od dve milijarde elemenata.

Zbog toga je spektralno klasterovanje, kao i hijerarhijsko, primenjeno nad stratifikovanim uzorkom od 10 000 igrača.

```
1 from sklearn.cluster import SpectralClustering
2
3 player_ids_original = player_ids.copy()
4 player_ids = player_ids_sample.copy()
5
6 top30_spectral_results = []
7
8 for k in [2, 3, 4, 5, 6, 7, 10]:
9     labels, res = run_algorithm(
10         X_top30_sample,
11         "top30_sample10000",
12         f"spectral_{k}",
13         SpectralClustering(
14             n_clusters=k,
15             random_state=42,
16             affinity="nearest_neighbors",
17             n_neighbors=10,
18             assign_labels="kmeans"
19         )
20     )
21     top30_spectral_results.append(res)
22
23 player_ids = player_ids_original.copy()
```

Listing 20: Spektralno klasterovanje nad uzorkom

Korišćenje grafa najbližih suseda smanjuje zahtevnost u odnosu na punu RBF matricu i bolje odgovara velikim skupovima.

12. DBSCAN

12.1. Osnovna ideja

DBSCAN je algoritam klasterovanja zasnovan na gustini.

Koristi dva glavna parametra:

ε maksimalno rastojanje između susednih tačaka;

min_samples minimalan broj tačaka potreban da bi se oblast smatrala dovoljno gustom.

Za tačku x_i , njeno ε -susedstvo definisano je kao:

$$N_\varepsilon(x_i) = \{x_j : d(x_i, x_j) \leq \varepsilon\}.$$

Tačka je jezgrena ako važi:

$$|N_\varepsilon(x_i)| \geq \text{min_samples}.$$

Tačke koje nisu deo nijedne dovoljno guste oblasti označavaju se kao šum, oznakom -1 .

12.2. Prednosti

DBSCAN:

- ne zahteva unapred zadat broj klastera;
- može pronaći klastere nepravilnog oblika;
- eksplicitno prepoznaje šumske instance;
- može otkriti male i specifične grupe igrača.

12.3. Ograničenja

Rezultati su veoma osetljivi na izbor parametara.

U prostoru velikog broja dimenzija rastojanja postaju manje informativna, što dodatno otežava izbor ε .

Mala promena vrednosti ε može dovesti do:

- spajanja velikog broja tačaka u jedan klaster;
- pojave desetina ili stotina malih klastera;
- značajnog povećanja broja šumskih instanci.

12.4. Ispitane vrednosti parametara

Ispitivane su kombinacije:

$$\varepsilon \in \{2.0, 2.5, 3.0, 3.5, 4.0\}$$

i:

$$\text{min_samples} \in \{5, 10, 20\}.$$

```
1 from sklearn.cluster import DBSCAN
2
3 top30_dbscan_results = []
4
5 eps_values = [2.0, 2.5, 3.0, 3.5, 4.0]
6 min_samples_values = [5, 10, 20]
7
8 for eps in eps_values:
9     for min_samples in min_samples_values:
10         labels, res = run_algorithm(
11             X_top30,
12             "top30",
13             f"dbscan_eps{eps}_min{min_samples}",
14             DBSCAN(
15                 eps=eps,
16                 min_samples=min_samples,
17                 n_jobs=-1
18             )
19         )
20
21         res["param"] = f"eps={eps}, min_samples={min_samples}"
22         top30_dbscan_results.append(res)
```

Listing 21: Pokretanje DBSCAN eksperimenata

12.5. Primeri dobijenih rezultata

Određene konfiguracije dale su sledeće rezultate:

	dataset	algorithm	n_clusters	n_noise	silhouette	davies_bouldin	calinski_harabasz	time_seconds	param
2	top30	dbscan_eps2.0_min20	4	44795	0.836545	0.227931	1545.597210	4.802134	eps=2.0, min_samples=20
5	top30	dbscan_eps2.5_min20	21	44140	0.621737	0.557091	680.393036	4.656730	eps=2.5, min_samples=20
1	top30	dbscan_eps2.0_min10	43	44018	0.615471	0.518106	621.368946	7.733962	eps=2.0, min_samples=10
8	top30	dbscan_eps3.0_min20	46	43219	0.527369	0.753301	541.788972	5.227837	eps=3.0, min_samples=20
4	top30	dbscan_eps2.5_min10	119	42435	0.512621	0.727439	461.084578	4.483177	eps=2.5, min_samples=10
0	top30	dbscan_eps2.0_min5	252	42223	0.508968	0.666925	371.004383	7.765723	eps=2.0, min_samples=5
11	top30	dbscan_eps3.5_min20	80	41405	0.464427	0.860642	455.652563	5.300107	eps=3.5, min_samples=20
3	top30	dbscan_eps2.5_min5	472	39338	0.437972	0.817956	283.123300	4.171511	eps=2.5, min_samples=5
7	top30	dbscan_eps3.0_min10	230	39978	0.423896	0.892377	346.147135	5.005268	eps=3.0, min_samples=10
14	top30	dbscan_eps4.0_min20	135	38130	0.419975	0.966258	389.875554	5.374076	eps=4.0, min_samples=20
10	top30	dbscan_eps3.5_min10	313	36766	0.377805	0.989956	288.403681	5.343375	eps=3.5, min_samples=10
6	top30	dbscan_eps3.0_min5	659	36101	0.372848	0.936966	226.909633	4.689659	eps=3.0, min_samples=5
13	top30	dbscan_eps4.0_min10	364	32362	0.344965	1.078759	261.411917	5.458238	eps=4.0, min_samples=10
9	top30	dbscan_eps3.5_min5	727	32152	0.314261	1.041186	192.023522	5.981960	eps=3.5, min_samples=5
12	top30	dbscan_eps4.0_min5	729	27820	0.290230	1.115961	173.747689	5.303982	eps=4.0, min_samples=5

Slika 2: Primer rezultata DBSCAN algoritma.

Rezultati pokazuju da smanjivanje parametra `min_samples` uglavnom dovodi do formiranja većeg broja malih klastera.

Konfiguracija sa $\varepsilon = 2.0$ i `min_samples=20` dala je visok `silhouette` rezultat i mali Davies–Bouldin indeks. Ipak, to ne znači automatski da je podela najbolja za interpretaciju. Potrebno je proveriti:

- veličine četiri dobijena klastera;
- da li jedan klaster sadrži većinu igrača;
- koji tipovi igrača pripadaju manjim klasterima;
- koliko rezultat zavisi od visoke dimenzionalnosti;
- da li su klasteri posledica binarnih kategorijskih atributa.

13. Detaljniji eksperimenti

U notebook `04_clustering.ipynb` može se videti da su algoritmi klasterovanja poređeni na svim izdvojenim skupovima podataka.

13.1. Klasterovanje nad različitim skupovima

Poređeni su rezultati nad:

- kompletnim skupom `X_full`;
- skupom `X_top30` - samo 30 najučestalijih zemalja;
- skupom `X_player_only` - samo igrački atributi;
- PCA skupom sa 90% varijanse dobijenim od prethodnog;

- kompletnim skupom, ali bez atributa o poreklu igrača
- prethodni skup, ali sa izbačenim klupskim atributima
- PCA skupom sa 95% varijanse nad prethodnim skupom;

Cilj nije bio samo pronaći najbolju vrednost metrika, već utvrditi koji skup daje:

- stabilne klastere;
- razumno uravnotežene veličine;
- sportsku interpretabilnost;
- prihvatljivo vreme izvršavanja.

13.2. Ograničavanje skupih algoritama

Hijerarhijsko i spektralno klasterovanje ponovo su ograničeni na 10 000 stratifikovano izabranih instanci.

K-sredina i GMM mogli su se primenjivati nad kompletnim skupovima, dok je DBSCAN pokretan selektivno zbog velikog vremena izvršavanja.

14. Evaluacija kvaliteta klasterovanja

Pošto ne postoje poznate ciljne klase, korišćene su interne mere kvaliteta klasterovanja.

14.1. Silhouette koeficijent

Za instancu i , neka je:

- $a(i)$ prosečno rastojanje do drugih instanci u istom klasteru;
- $b(i)$ najmanje prosečno rastojanje do instanci nekog drugog klastera.

Silhouette vrednost instance je:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

Važi:

$$-1 \leq s(i) \leq 1.$$

Tumačenje:

- vrednost bliska 1 ukazuje na dobro odvojenu instancu;

- vrednost bliska 0 ukazuje na graničnu instancu;
- negativna vrednost ukazuje da je instanca možda dodeljena pogrešnom klasteru.

Ukupan rezultat je prosek preko svih posmatranih instanci.

Veća vrednost je poželjnija.

14.2. Davies–Bouldin indeks

Za klastera C_i i C_j posmatra se odnos njihove unutrašnje rasutosti i rastojanja centroida:

$$R_{ij} = \frac{S_i + S_j}{M_{ij}},$$

gde su:

- S_i prosečno rastojanje instanci klastera i od njegovog centroida;
- M_{ij} rastojanje između centroida klastera i i j .

Davies–Bouldin indeks je:

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} R_{ij}.$$

Manja vrednost je poželjnija.

14.3. Calinski–Harabasz indeks

Calinski–Harabasz indeks predstavlja meru kvaliteta klasterovanja koja poredi međuklasterku razdvojenost i unutar-klasterku kompaktnost.

Definiše se formulom:

$$CH = \frac{\text{tr}(B)}{\text{tr}(W)} \cdot \frac{n - K}{K - 1},$$

gde su:

- n ukupan broj instanci;
- K broj klastera;
- W matrica unutar-klasterke rasutosti;
- B matrica između-klasterke rasutosti.

Veća vrednost Calinski–Harabasz indeksa ukazuje na bolje klasterovanje, odnosno na kompaktnije klastera koji su međusobno bolje razdvojeni.

Za razliku od silhouette koeficijenta, ovaj indeks nema ograničen opseg vrednosti, pa se koristi prvenstveno za poređenje različitih modela i konfiguracija nad istim skupom podataka.

14.4. Dodatni kriterijumi

Pored numeričkih metrika posmatrani su:

- broj klastera;
- šum;
- veličine klastera;
- odnos najvećeg i najmanjeg klastera;
- raspodela pozicija po klasterima;
- prosečne vrednosti najvažnijih numeričkih atributa;
- vreme izvršavanja;
- mogućnost interpretacije.

14.5. Zašto jedna metrika nije dovoljna

Visok silhouette koeficijent može biti posledica toga što je nekoliko malih, veoma udaljenih grupa izdvojeno od jednog dominantnog klastera.

Slično, mali Davies–Bouldin indeks ne garantuje da klasteri imaju praktično sportsko značenje.

Zato je izbor najboljeg rešenja zasnovan na kombinaciji:

$$\text{kvalitet} = \text{metrike} + \text{uravnoteženost} + \text{interpretabilnost}$$

15. Interpretacija klastera

Interpretacija klastera obrađena je u notebook-u `05_cluster_analysis`. Za potrebe interpretacije poređeni su rezultati klasterovanja dobijeni u prethodnom notebook-u, a zatim je od svih algoritama za svaki skup odabran onaj koji je davao najbolje rezultate. Nad klasterima je sprovedena jedna ustanovljena analiza koja je izdvajala igrače prema raznim karakteristikama, kao što su:

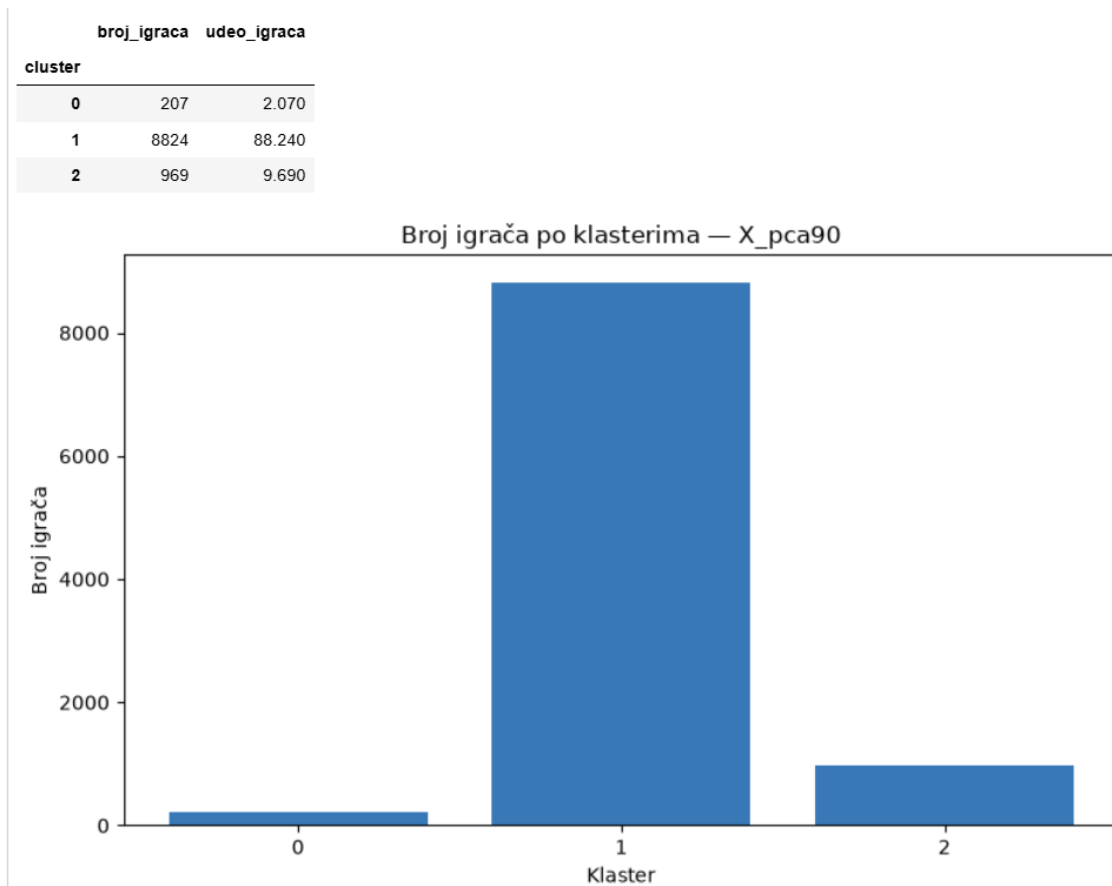
- pozicija;
- godine;
- broj odigranih utakmica;

- minutaža;
- koliko puta je igrač bio u startnoj postavi;
- koliko puta je ušao kao izmena;
- koliko puta je bio kapiten;
- broj nastupa u reprezentaciji;
- broj golova;
- broj asistencija;
- kao i drugi atributi.

Pre prolaska kroz primer, bitno je naglasiti da je samo klasterovanje na ovim skupovima dosta nezahvalan posao jer se radi o skupovima visoke dimenzionalnosti, visoke zavisnosti među podacima i visoke granularnosti podataka. Iz tog razloga rezultati su najbolji koje je moguće dobiti. Svakako da se specifičnijim klasterovanjem mogu dobiti bolji, a to je nešto što ćemo videti i u nastavku, kada ćemo izdvajati najbolje igrače po onome koliko se isplati investirati u njih. Tada klasterovanje ima više smisla i daje bolje rezultate.

Pokretanjem ovog notebook-a se mogu videti rezultati analize klasterovanja. Ovde će biti naveden samo primer u kojem se analizirao skup `X_pca90`. Nad ovim skupom pokretan je algoritam hijerarhijskog klasterovanja sa 3 klastera. Budući da hijerarhijsko klasterovanje ima ozbiljne računarske zahteve, kao što je i ranije spomenuto, i ovde je korišćen stratifikovani uzorkovani skup od 10 000 instanci.

Analiza daje sledeće rezultate:



Slika 3: Raspodela igrača po klasterima.

Prosečne vrednosti atributa po klasterima:

cluster	age	height_in_cm	international_caps	international_goals	market_value_in_eur	highest_market_value_in_eur	matches_played	total_minutes	minutes
0	26.614	183.473	28.420	6.609	27,547,584.541	44,347,826.087	193.643	13,282.986	27
1	29.103	182.222	1.934	0.146	574,199.909	1,292,634.859	22.334	1,501.433	12
2	34.176	181.576	10.970	1.157	2,752,167.183	10,618,575.851	179.946	12,726.596	25

Medijane atributa po klasterima:

cluster	age	height_in_cm	international_caps	international_goals	market_value_in_eur	highest_market_value_in_eur	matches_played	total_minutes	minutes
0	25.470	184.000	12.000	1.000	22,000,000.000	35,000,000.000	157.000	10,258.000	28
1	28.240	182.000	0.000	0.000	200,000.000	450,000.000	2.000	69.500	0
2	34.090	182.000	0.000	0.000	800,000.000	7,000,000.000	172.000	11,741.000	27

Slika 4: Prosek i medijana atributa po klasterima.

Raspodela atributa 'position' po klasterima (%):

position	Attack	Defender	Goalkeeper	Midfield	Missing
cluster					
0	39.610	26.090	3.860	30.430	0.000
1	26.560	31.310	12.890	28.150	1.090
2	28.790	37.770	0.310	33.130	0.000

Slika 5: Raspodela pozicija po klasterima.

Raspodela atributa 'sub_position' po klasterima (%):

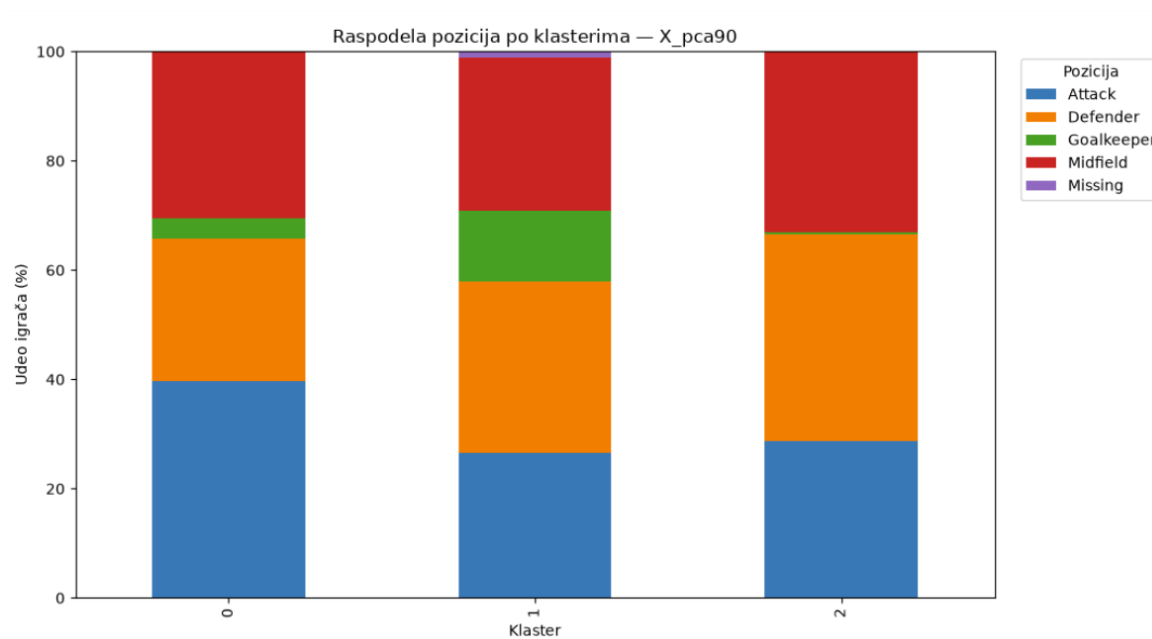
sub_position	Attacking Midfield	Central Midfield	Centre- Back	Centre- Forward	Defensive Midfield	Goalkeeper	Left Midfield	Left Winger	Left- Back	Right Midfield	Right Winger	Right- Back	Second Striker	Unknown
cluster														
0	5.800	16.430	9.660	24.640	8.210	3.860	0.000	6.760	5.800	0.000	5.800	10.630	2.420	0.000
1	6.940	11.020	17.320	13.510	7.840	12.890	1.290	6.330	6.740	1.070	6.090	7.250	0.630	1.090
2	7.020	12.280	20.740	11.870	12.280	0.310	0.930	8.670	8.880	0.620	8.050	8.150	0.210	0.000

Slika 6: Raspodela preciznijih pozicija po klasterima.

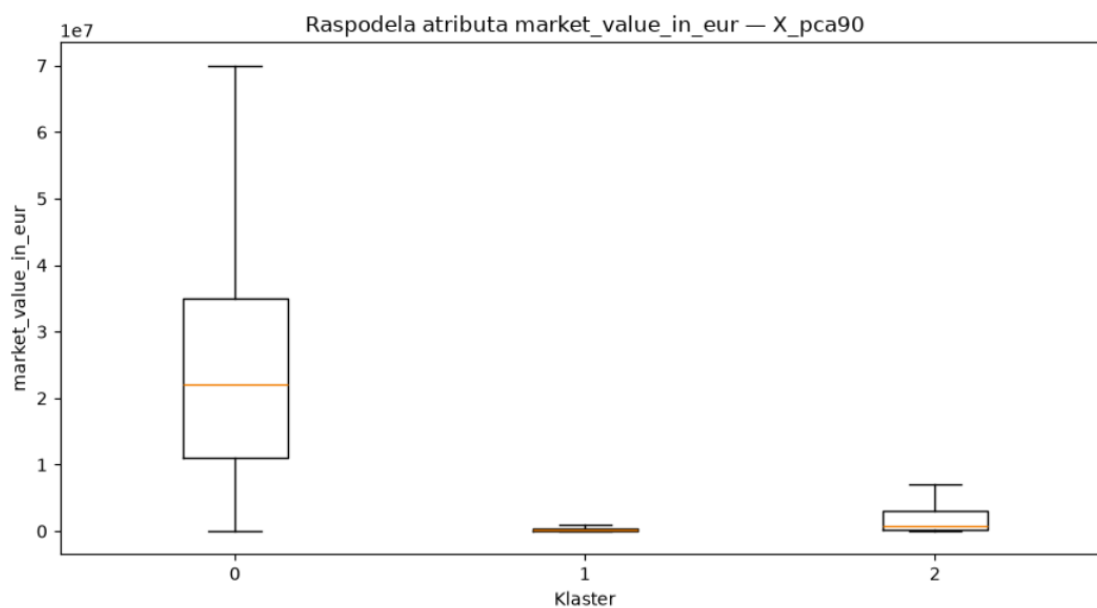
Raspodela atributa 'foot' po klasterima (%):

foot	Unknown	both	left	right
cluster				
0	0.000	4.830	25.600	69.570
1	11.840	3.860	22.140	62.150
2	1.140	2.680	26.930	69.250

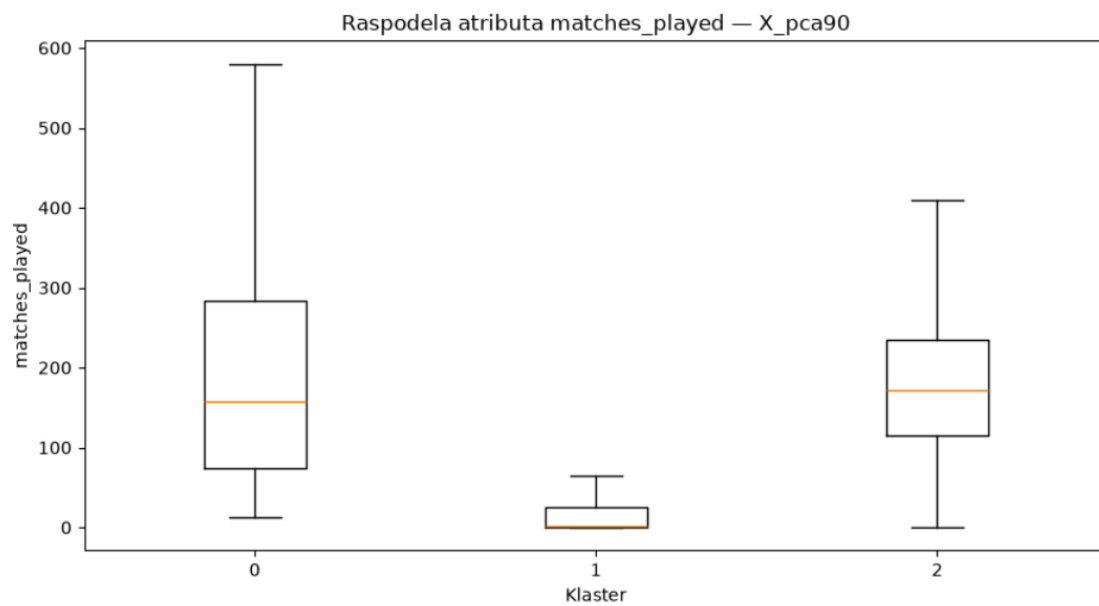
Slika 7: Prikaz koja noga je dominantna u kojem klasteru.



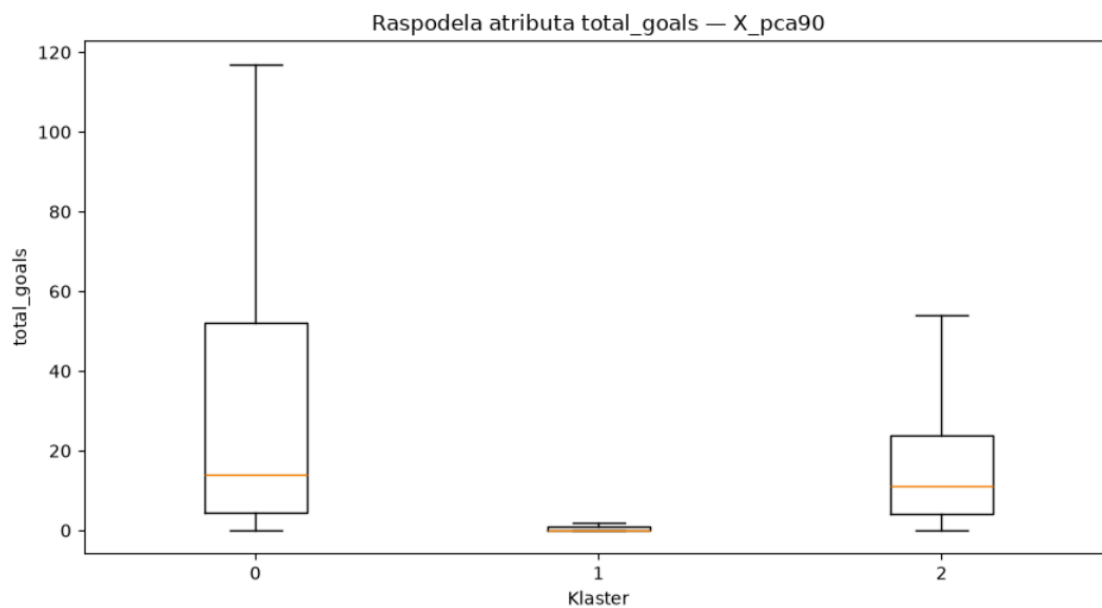
Slika 8: Raspodela pozicija po klasterima.



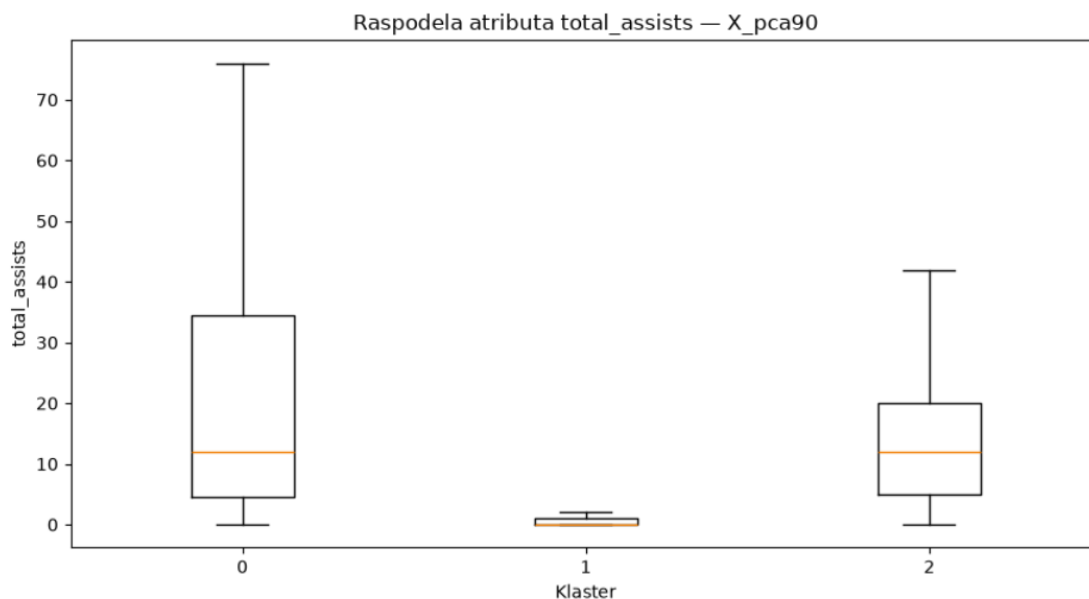
Slika 9: Raspodela tržišne vrednosti po klasterima.



Slika 10: Raspodela odigranih utakmica po klasterima.



Slika 11: Raspodela golova po klasterima.



Slika 12: Raspodela asistencija po klasterima.

Na osnovu prikazanih rezultata, a najviše poslednjih nekoliko grafika, možemo videti jasnu razliku između pronađenih klastera. U prvom klasteru se radi o igračima koji igraju puno utakmica, imaju najviše minuta, golova i asistencija, pa zbog toga i ubedljivo najveću tržišnu vrednost. Drugi i najbrojniji klaster sadrži igrače koji nemaju velike minutaže, nemaju velike udele u postignutim golovima, pa tako ni velike tržišne vrednosti. Ovde se može raditi o lošijim igračima ili o mladim igračima koji još nisu stekli pravu šansu za dokazivanjem. U trećem klasteru možemo videti da se nalaze igrači sa visokim minutažama, nešto višim udelima u golovima i asistencija, ali sa izrazito niskim tržišnim vrednostima. Ovde se može pretpostaviti da se radi o veteranima, ali i o igračima koji igraju u slabijim ligama, a mogu zapravo predstavljati dobru investiciju prilikom skautinga.

15.1. VFM skup

Na osnovu prethodne analize stvorila se potreba za izdvajanjem igrača koji bi mogli predstavljati potencijalno dobre investicije za klub. U tom svetlu, napravljen je poseban skup podataka - `X_value_for_money`. U ovom skupu nalaze se samo najpotrebnije informacije o igračima, konkretno one koje su direktno bitne prilikom skautovanja novih igrača, kao što su minutaža, vrednost, golovi, asistencije, pozicija i tako dalje. Ovom skupu je posvećena posebna pažnja, a rezultati analize se mogu ponoviti pokretanjem notebook-a `06_more_specific_clustering`.

Nad skupom je najpre pokrenuto svih 5 algoritama klasterovanja. Na osnovu rezultata, odlučeno je da se analiza nastavi nad onime što je dobijeno KMeans algoritmom sa

3 klastera. Na osnovu raspodele pozicija igrača u klasterima, ustanovljeno je da se u jednom klasteru nalaze isključivo golmani.

position	Attack	Defender	Goalkeeper	Midfield	Missing
cluster					
0	0.000	0.000	1.000	0.000	0.000
1	0.307	0.356	0.000	0.325	0.013
2	0.291	0.371	0.026	0.312	0.000

Slika 13: Raspodela pozicija po klasterima na Value-for-Money skupu.

Dalje, analizom prosečne vrednosti igrača u klasterima, ustanovljeno je da se u jednom od dva preostala klastera nalaze afirmisani igrači, dok se u drugom nalaze lošiji, ali i mlađi igrači.

avg_market_value_millions	median_market_value_millions	avg_highest_market_value_millions
0.51	0.1	1.09
0.79	0.2	1.46
4.97	0.8	13.07

Slika 14: Raspodela vrednosti igrača po klasterima.

Dalja analiza se odnosila na svaki od ovih klastera ponaosob. Ono što je zajedničko za sve klastere jeste da se nastojalo da se pronađu igrači koji mogu biti dobre potencijalne investicije za klub, odnosno igrači koji imaju najbolji odnos cene i kvaliteta. U prvom klasteru je sprovedena ova vrsta analize nad golmanima, u drugom klasteru nad mlađim pripadnicima klastera, a u poslednjem klasteru nad već afirmisanim igračima.

15.1.1. Golmani

Za golmane su se ispitivali sledeći uslovi:

- starost manja od 30 godina
- više od 30 odigranih utakmica

- više od 2000 minuta provedenih na terenu

Kako se ne bi dobijali neočekivani rezultati, uveden je i uslov da se gledaju samo golmani čija je cena strogo veća od 0.

Nakon izdvajanja golmana po navedenim uslovima, rezultati su sortirani najpre po procentu porasta vrednosti, zatim po nastupima za reprezentaciju i, na kraju, po ukupnom broju odigranih minuta. Dobijeni su sledeći rezultati:

	name	age	market_value_in_eur	highest_market_value_in_eur	international_caps	matches_played	total_minutes	starter_ratio	valuation_growth_pct
0	Bart Verbruggen	23.82	40000000.0	40000000.0	27.0	115.0	10410.0	0.500000	1599.000000
1	Giorgi Mamardashvili	25.70	28000000.0	45000000.0	37.0	147.0	13177.0	0.808989	1119.000000
2	Noah Atubolu	24.05	20000000.0	20000000.0	22.0	113.0	10153.0	0.770642	799.000000
3	Joan García	25.11	40000000.0	40000000.0	0.0	82.0	7329.0	0.529801	799.000000
4	Guglielmo Vicario	29.68	23000000.0	35000000.0	5.0	194.0	17467.0	0.790476	459.000000
5	Zion Suzuki	23.81	20000000.0	20000000.0	22.0	74.0	6645.0	0.984615	399.000000
6	Berke Özer	26.05	10000000.0	10000000.0	2.0	95.0	8413.0	0.574074	399.000000
7	Robin Roefs	23.40	25000000.0	25000000.0	0.0	71.0	6389.0	0.340909	332.333333
8	Julen Agirrezabala	25.46	15000000.0	15000000.0	8.0	84.0	7488.0	0.446927	299.000000
9	Senne Lammens	23.93	30000000.0	30000000.0	2.0	63.0	5657.0	0.253425	299.000000
10	Filip Jørgensen	24.16	15000000.0	20000000.0	1.0	60.0	5349.0	0.363636	299.000000
11	Elia Caprile	24.80	15000000.0	15000000.0	0.0	80.0	7164.0	0.505155	299.000000
12	Altay Bayındır	28.16	7000000.0	16000000.0	11.0	168.0	15162.0	0.614815	279.000000
13	Djordje Petrovic	26.68	28000000.0	28000000.0	11.0	95.0	8526.0	0.779070	279.000000
14	Dean Henderson	29.25	28000000.0	28000000.0	4.0	176.0	15751.0	0.620000	279.000000
15	Kjell Scherpen	26.38	7000000.0	7000000.0	0.0	103.0	9194.0	0.657303	279.000000
16	Moritz Nicolas	28.64	6000000.0	6000000.0	1.0	82.0	7340.0	0.490741	239.000000
17	Maduka Okoye	26.79	10000000.0	10000000.0	19.0	130.0	11611.0	0.801471	199.000000
18	Stanislav Agkatsev	24.42	10000000.0	10000000.0	4.0	84.0	7507.0	0.401408	199.000000
19	Mads Hermansen	25.92	15000000.0	18000000.0	1.0	90.0	8115.0	0.689655	199.000000

Slika 15: Rezultati izdvajanja golmana po navedenim uslovima.

Bitno je napomenuti da dobijeni rezultati ne znače da je golman na recimo 3. poziciji strogo bolji od golmana na recimo 11. poziciji. Svi ovi golmani su dobri i mogu predstavljati potencijalno dobre investicije za klub. Sama odluka zavisi od taktike kluba, njihovog budžeta, ali naravno i od toga kako taj golman zapravo igra (što se može jedino videti uživo), kao i od njegove ličnosti i procene koliko dobro bi se uklopio u ostatak ekipe.

15.1.2. Srednji klaster

Kao što je rečeno, ovaj klaster se sastoji od lošijih igrača, onih koji su većinu karijere proveli u slabijim ligama, veterana, ali i od mladih igrača koji tek čekaju svoju šansu. Analiza ovog klastera je upravo posvećena ovoj poslednjoj grupi igrača. Iz tog razloga, iz klastera su izdvojeni igrači na osnovu uslova da su mlađi od 22 godine, da su odigrali

više od 20 utakmica i proveli više od 1000 minuta na terenu. Dobijeni su sledeći rezultati:

	name	age	market_value_in_eur	highest_market_value_in_eur	valuation_growth_pct	starter_ratio	matches_played	total_minutes	international_caps
0	Jesús Rodríguez	20.56	30000000.0	30000000.0	1199.000000	0.523810	54.0	3072.0	1.0
1	El Hadji Malick Diouf	21.45	28000000.0	28000000.0	1119.000000	0.794118	26.0	2187.0	17.0
2	Jérémy Jacquet	20.91	55000000.0	55000000.0	1099.000000	0.640000	37.0	2855.0	5.0
3	Givairo Read	20.03	25000000.0	25000000.0	999.000000	0.500000	51.0	3558.0	1.0
4	Anan Khalaili	21.77	18000000.0	18000000.0	719.000000	0.615385	71.0	5162.0	16.0
5	Valentin Barco	21.89	35000000.0	35000000.0	699.000000	0.635135	56.0	4118.0	1.0
6	Ruben van Bommel	21.86	12000000.0	12000000.0	479.000000	0.658228	75.0	4375.0	16.0
7	Michael Kayode	21.92	35000000.0	35000000.0	465.666667	0.421053	87.0	6205.0	9.0
8	Jaydee Canvot	19.87	20000000.0	20000000.0	399.000000	0.600000	37.0	2259.0	6.0
9	Matvey Kislyak	20.88	20000000.0	20000000.0	399.000000	0.395349	75.0	5533.0	8.0
10	Rosen Bozhinov	21.38	4000000.0	4000000.0	399.000000	0.033333	28.0	1732.0	4.0
11	Sascha Britschgi	19.79	10000000.0	10000000.0	399.000000	0.000000	27.0	2084.0	5.0
12	Ezechiel Banzuzi	21.32	18000000.0	18000000.0	359.000000	0.709677	82.0	4549.0	12.0
13	Samir El Mourabet	20.68	18000000.0	18000000.0	359.000000	0.464286	27.0	1754.0	1.0
14	Santiago Castro	21.73	35000000.0	35000000.0	349.000000	0.621622	95.0	5769.0	0.0
15	Said El Mala	19.79	35000000.0	40000000.0	349.000000	0.000000	29.0	1467.0	6.0
16	Christos Mouzakitis	19.46	25000000.0	25000000.0	332.333333	0.617021	65.0	3926.0	8.0
17	Lery Yoro	20.58	50000000.0	55000000.0	332.333333	0.560976	108.0	7279.0	9.0
18	Charalampos Kostoulas	19.04	25000000.0	25000000.0	332.333333	0.488372	55.0	2339.0	5.0
19	Anthony Dennis	21.97	8000000.0	8000000.0	319.000000	1.000000	56.0	4622.0	0.0

Slika 16: Rezultati izdvajanja mladih igrača po navedenim uslovima.

Igrači su zatim sortirani ponovo po procentualnom rastu vrednosti, minutaži, nastupima u reprezentaciji, ali i po količini utakmica u kojima su bili u startnoj postavi. Naravno, kao i kod golmana, najbolji igrač na ovoj tabeli ne znači nužno da on predstavlja i najbolju investiciju.

15.1.3. Afirmisani igrači

Treći klaster se sastoji od već afirmisanih igrača, onih koji su standardni starteri u svojim ekipama, imaju veliki broj nastupa za reprezentaciju, veliki broj minuta, kao i od elitnih, najpopularnijih igrača. Budući da se za većinu ovih igrača uveliko zna i da je dosta njih već popularno, u ovoj kategoriji je odlučeno da se za dobre investicije smatraju igrači koji su mlađi od 30 godina i koji pripadaju donjoj polovini igrača po tržišnoj vrednosti. Dobijeni su sledeći rezultati:

name	age	position	market_value_in_eur	highest_market_value_in_eur	international_caps	matches_played	total_minutes	starter_ratio	valuation_growth_pct
Otabek hukurov	29.97	Midfield	1600000.0	4000000.0	85.0	55.0	3025.0	0.696203	4.818182
istopher Martins	29.31	Midfield	4500000.0	8000000.0	80.0	144.0	8643.0	0.610294	44.000000
Grigoris astanos	28.36	Midfield	1800000.0	1800000.0	76.0	126.0	5612.0	0.357542	23.000000
el Sinani	29.19	Midfield	3000000.0	3000000.0	71.0	75.0	4404.0	0.520408	119.000000
osé Luis idriguez	27.98	Attack	2500000.0	2500000.0	66.0	32.0	2122.0	0.640000	24.000000
Rashica	29.85	Attack	4000000.0	35000000.0	63.0	350.0	24274.0	0.785100	79.000000
Oleg eaboiuk	28.40	Defender	3000000.0	5000000.0	62.0	207.0	15826.0	0.752252	119.000000
Shamar holson	29.24	Attack	3200000.0	7500000.0	62.0	164.0	9335.0	0.557895	63.000000
Mostafa phamed	28.54	Attack	4000000.0	10000000.0	61.0	185.0	9418.0	0.502564	159.000000
Kalechi ianacho	29.69	Attack	3000000.0	20000000.0	58.0	264.0	11324.0	0.323944	29.000000
Ion olaescu	27.76	Attack	1000000.0	1500000.0	56.0	50.0	2274.0	0.328767	19.000000
Róbert Boženič	26.57	Attack	3500000.0	5000000.0	55.0	112.0	7466.0	0.600000	69.000000
Samuel ussamy	29.83	Midfield	1200000.0	5000000.0	55.0	225.0	13268.0	0.476190	5.000000
Jorge ánchez	28.50	Defender	2200000.0	6000000.0	55.0	54.0	2999.0	0.417722	1.200000
n Dagur einsson	27.54	Attack	800000.0	3500000.0	53.0	164.0	10533.0	0.779762	1.666667
Angelo reciado	28.31	Defender	3800000.0	5000000.0	53.0	81.0	4570.0	0.514019	75.000000
Chris lepham	28.60	Defender	3500000.0	13000000.0	53.0	65.0	4887.0	0.509259	34.000000
Zuriko rtashvili	25.32	Attack	3000000.0	3000000.0	52.0	81.0	6071.0	0.812500	29.000000
Show	27.27	Midfield	1500000.0	2000000.0	52.0	79.0	5137.0	0.585859	2.750000
Kaboré	25.08	Defender	4000000.0	10000000.0	52.0	127.0	7433.0	0.441341	39.000000

Slika 17: Rezultati izdvajanja afirmisanih igrača po navedenim uslovima.

16. Vizualizacija rezultata

Vizualizacije su realizovane u notebook-u `07_cluster_visualization.ipynb`.

16.1. Ciljevi vizualizacije

Vizualizacije su korišćene za:

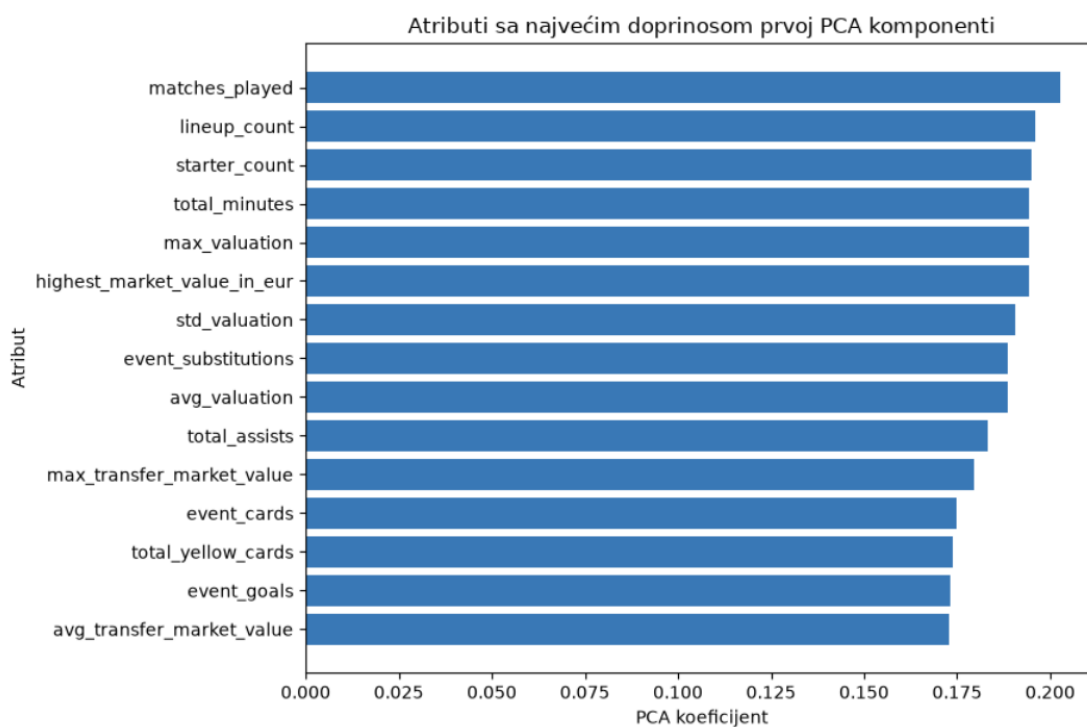
- prikaz prostorne razdvojenosti klastera;
- identifikovanje preklapanja;
- poređenje različitih algoritama;
- interpretaciju u odnosu na originalne attribute;
- prikaz raspodele veličina klastera.

Vizualizacija je sprovedena na osnovu rezultata hijerarhijskog i KMeans klasterovanja nad skupom `X_no_countries_no_club_pca95` i nad skupom `X_value_for_money`, odnosno njegovom PCA verzijom.

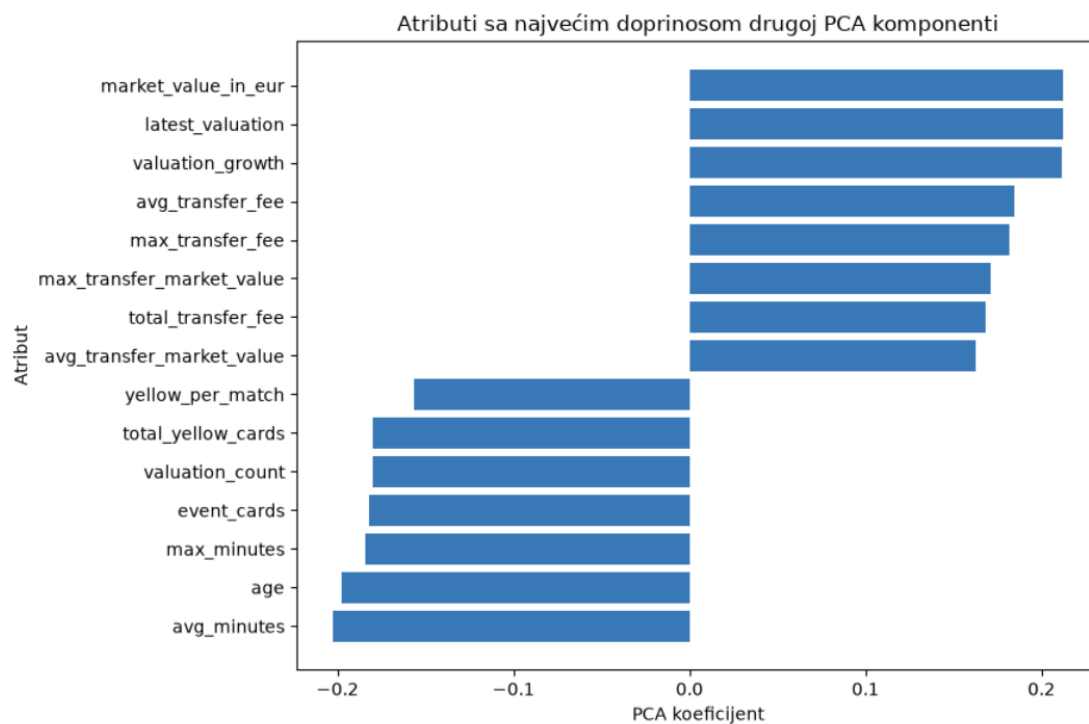
16.2. Analiza doprinosa originalnih atributa atributima nastalim PCA metodom

Nakon pokretanja PCA metode nad skupovima, najpre je ispitano kako izgledaju novodobijeni atributi. Budući da su nam za crtanje potrebne 3 dimenzije, ispitane su karakteristike prve 3 PCA komponente: PCA1, PCA2 i PCA3.

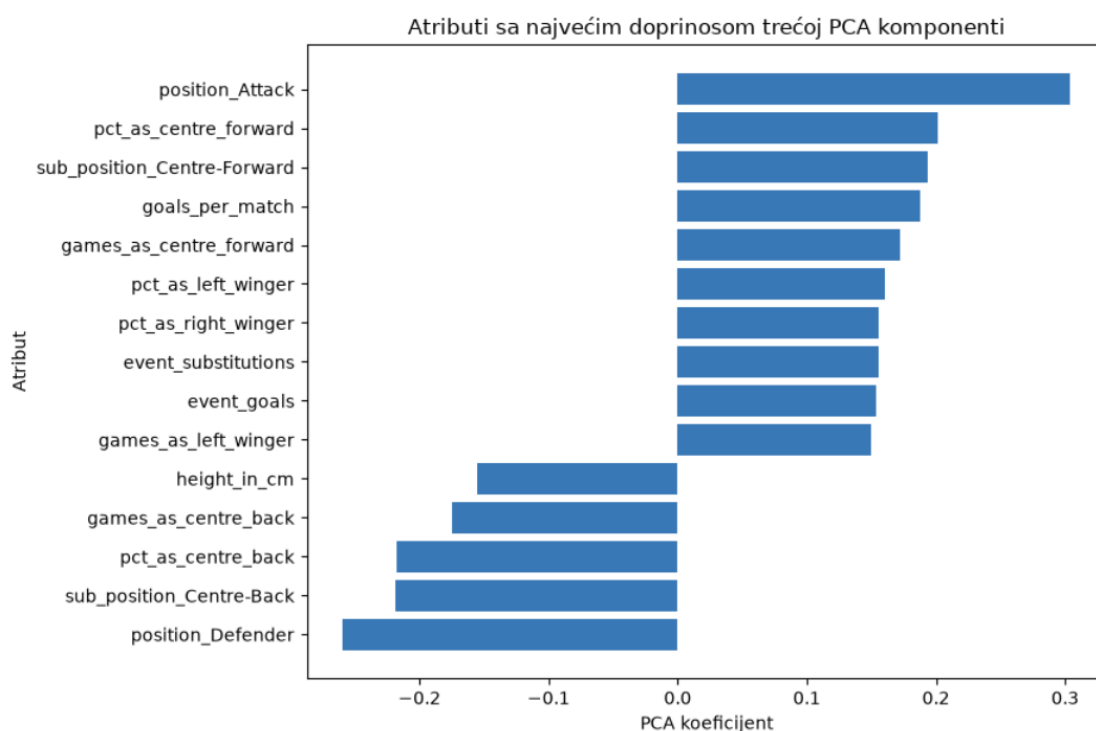
16.2.1. X_no_countries_no_club_pca95



Slika 18: Udeo originalnih atributa u prvoj PCA komponenti.



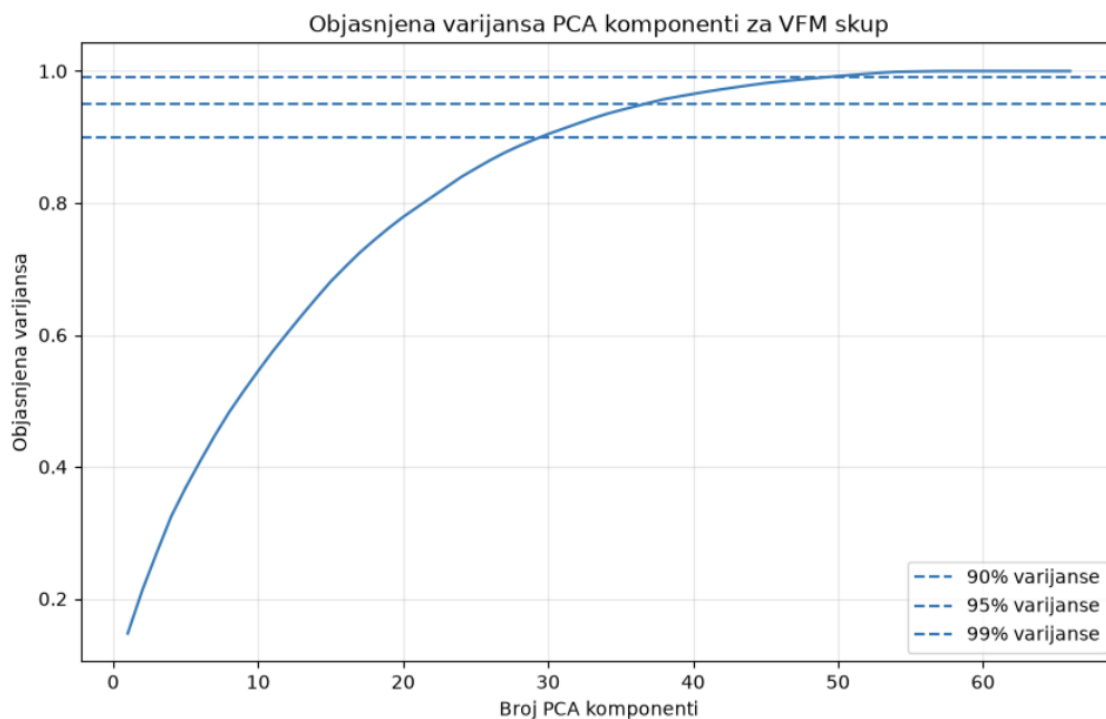
Slika 19: Udeo originalnih atributa u drugoj PCA komponenti.



Slika 20: Udeo originalnih atributa u trećoj PCA komponenti.

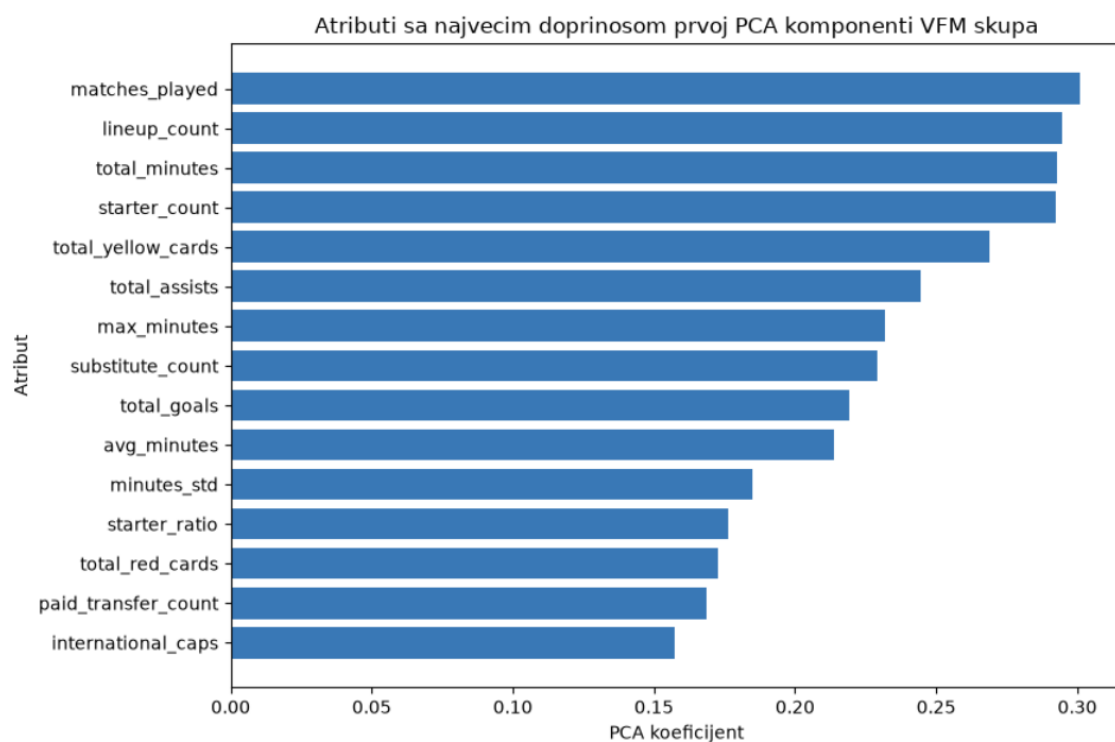
16.2.2. `x_value_for_money`

Ovde je najpre bilo potrebno sprovesti PCA metodu nad originalnim skupom podataka. Prilikom toga, ponovo je određivano koliko atributa je potrebno za zadržavanje određenog procenta varijanse, što se može videti na sledećem grafiku.

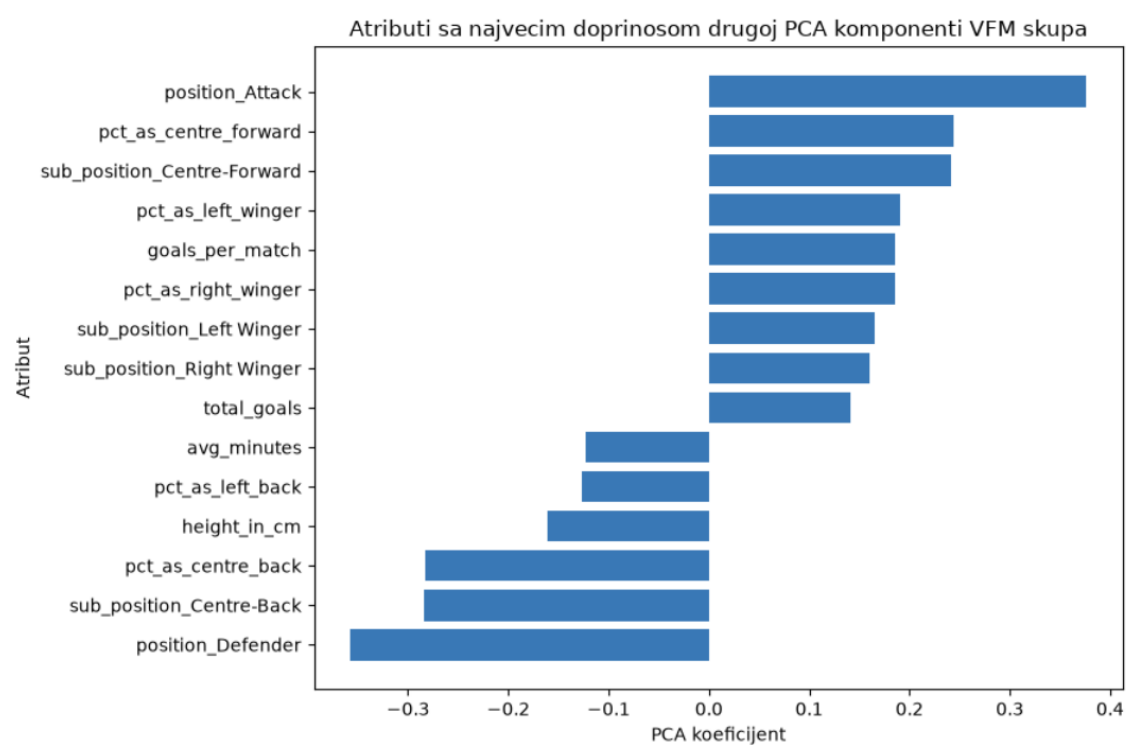


Slika 21: Grafik zavisnosti objašnjene varijanse u odnosu na broj atributa.

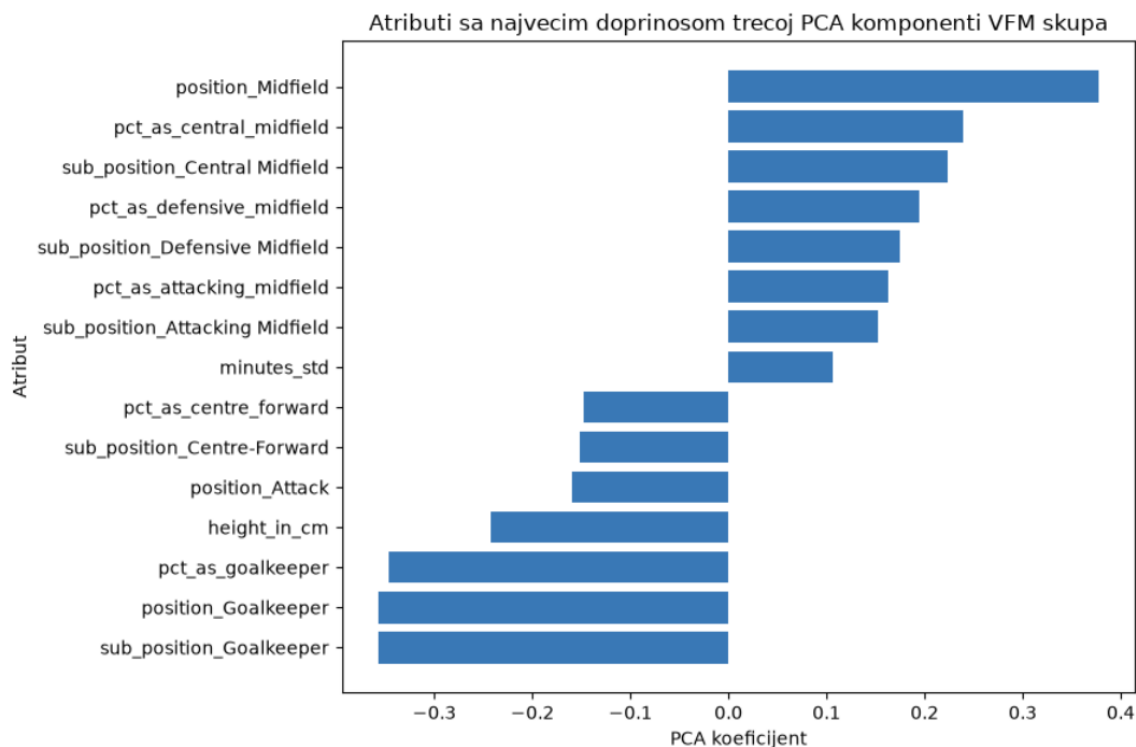
Može se primetiti da je za zadržavanje 90% varijanse potrebno 30 atributa, dok je za 95% potrebno njih 37. Budući da smanjenje od 7 atributa nije značajno, odlučeno je da se nastavi sa pristupom koji čuva 95% varijanse. Kao i prethodnom primeru, analiziramo udeo originalnih atributa u prve 3 PCA komponente.



Slika 22: Udeo originalnih atributa u prvoj PCA komponenti.



Slika 23: Udeo originalnih atributa u drugoj PCA komponenti.



Slika 24: Udeo originalnih atributa u trećoj PCA komponenti.

16.3. Dvodimenzionalna PCA projekcija

Za prikaz podataka u dve dimenzije korišćene su prve dve PCA komponente, koje ujedno i najviše utiču na očuvanje varijanse u podacima. Sledeći kod prikazuje primer 2D prikaza KMeans klasterovanja nad prvim navedenim skupom.

```

1 import matplotlib.pyplot as plt
2
3 plt.figure(figsize=(10, 8))
4
5 scatter = plt.scatter(
6     X_pca95["PC1"],
7     X_pca95["PC2"],
8     c=kmeans_labels,
9     cmap="tab10",
10    s=8,
11    alpha=0.6
12 )
13
14 plt.xlabel("PC1")
15 plt.ylabel("PC2")
16 plt.title("KMeans klasterovanje u 2D PCA prostoru")
17

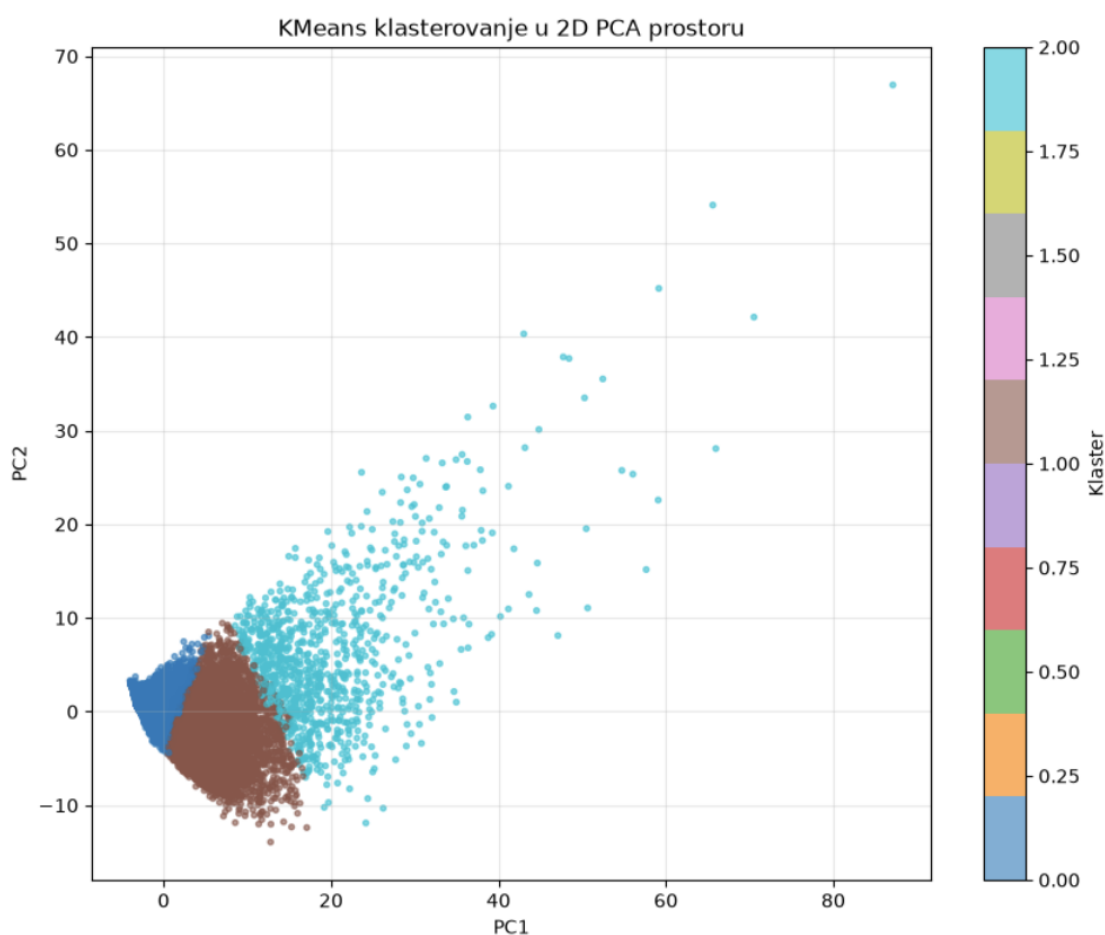
```

```
18 plt.colorbar(  
19     scatter,  
20     label="Klaster"  
21 )  
22  
23 plt.grid(alpha=0.3)  
24 plt.show()
```

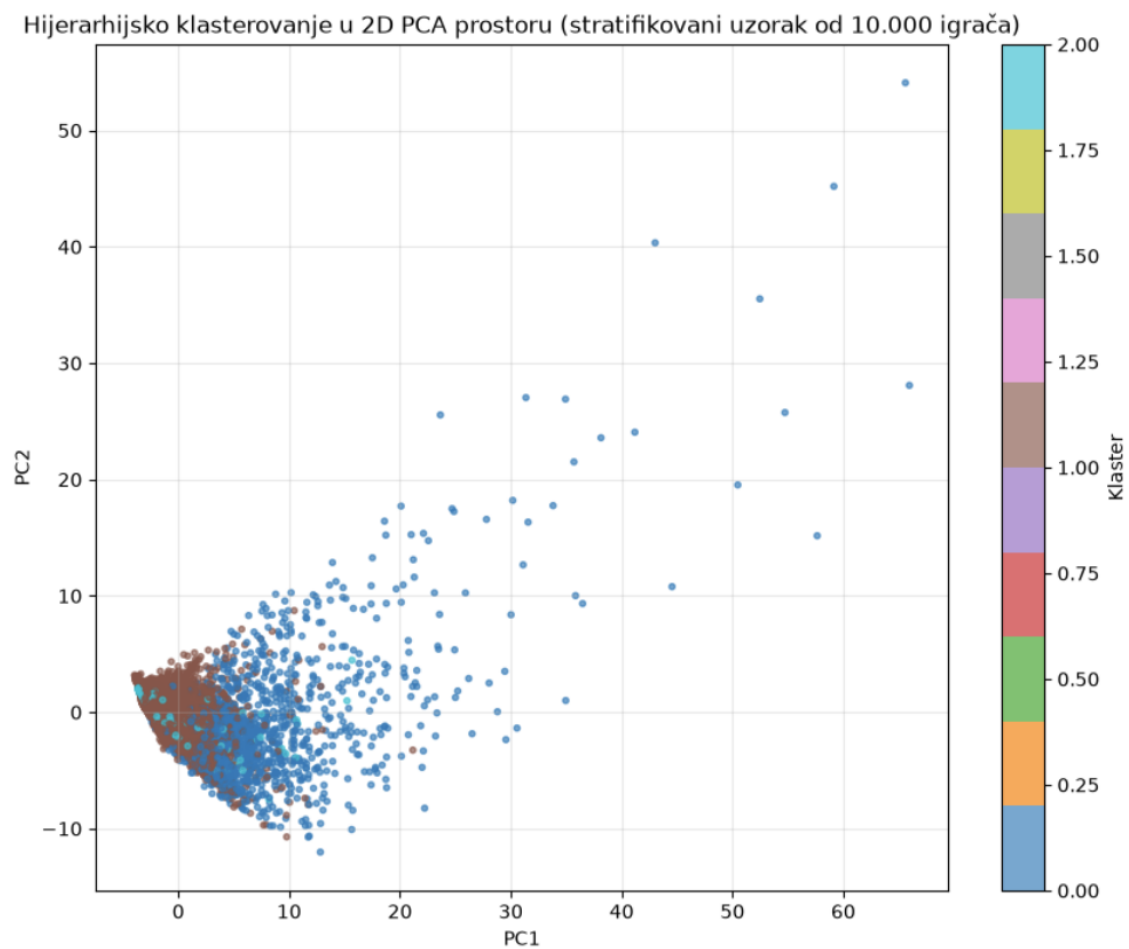
Listing 22: Dvodimenzionalna PCA projekcija

Važno je naglasiti da PCA projekcija na dve dimenzije ne čuva svu informaciju korišćenu pri klasterovanju. Dva klastera koja se preklapaju na slici mogu biti bolje razdvojena u višedimenzionalnom prostoru.

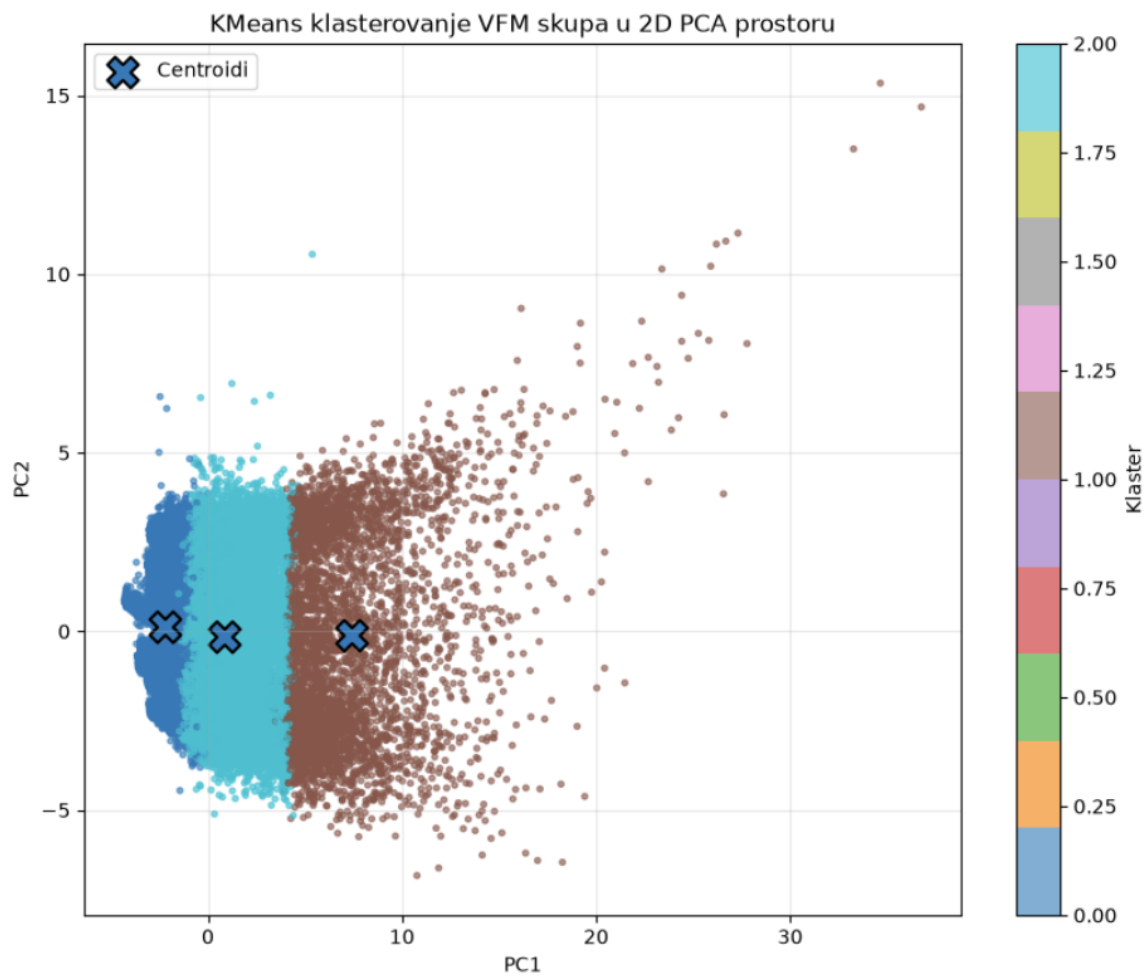
16.3.1. `x_no_countries_no_club`



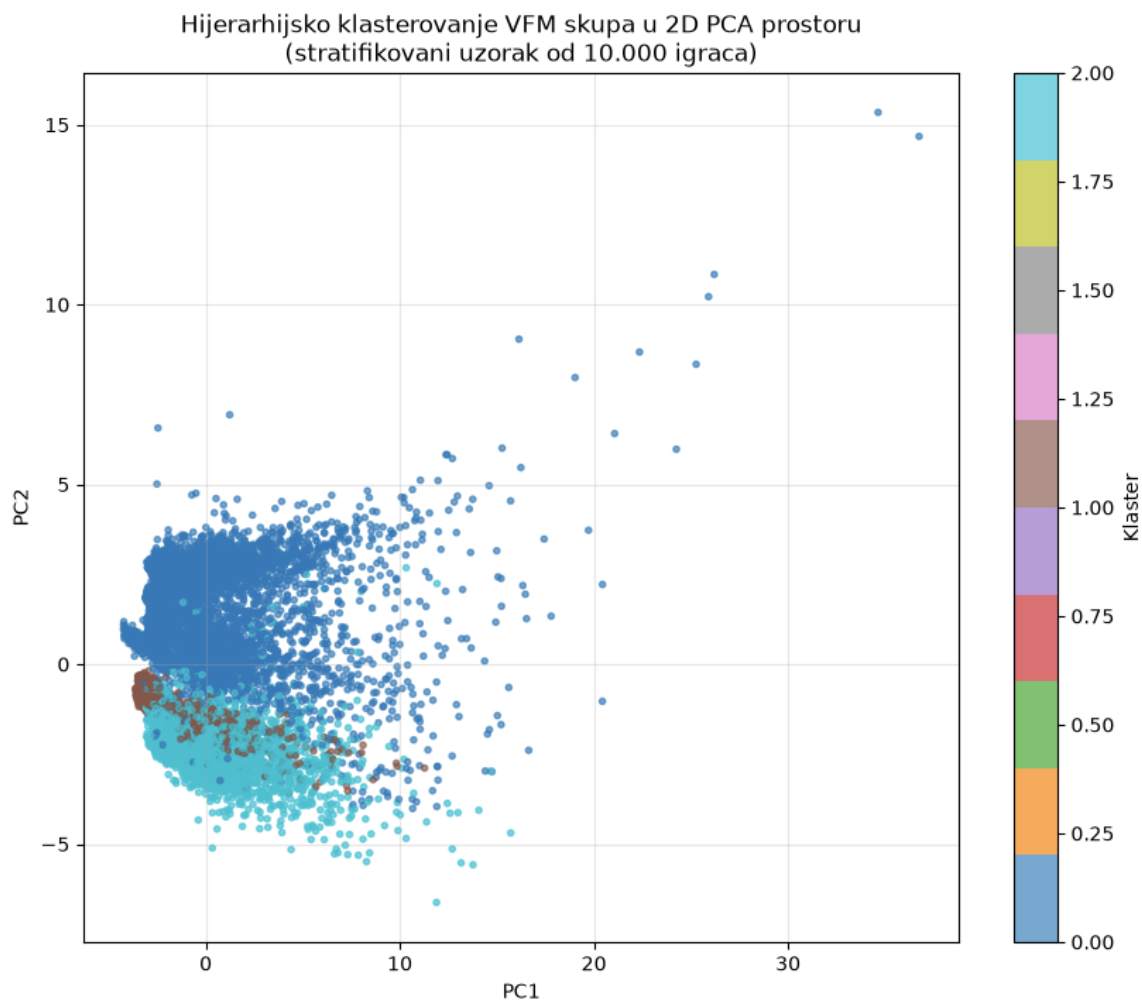
Slika 25: 2D vizualizacija klastera dobijenih KMeans algoritmom.



Slika 26: 2D vizualizacija klastera dobijenih hijerarhijskim algoritmom.

16.3.2. `x_value_for_money`

Slika 27: 2D vizualizacija klastera dobijenih KMeans algoritmom.



Slika 28: 2D vizualizacija klastera dobijenih hijerarhijskim algoritmom.

16.4. Trodimenzionalna PCA projekcija

Za dodatni prikaz korišćene su prve tri glavne komponente.

Ispod se nalazi primer koda korišćenog za 3D prikaz klastera, ovde konkretno rezultata KMeans algoritma nad prvim skupom, kao i iznad.

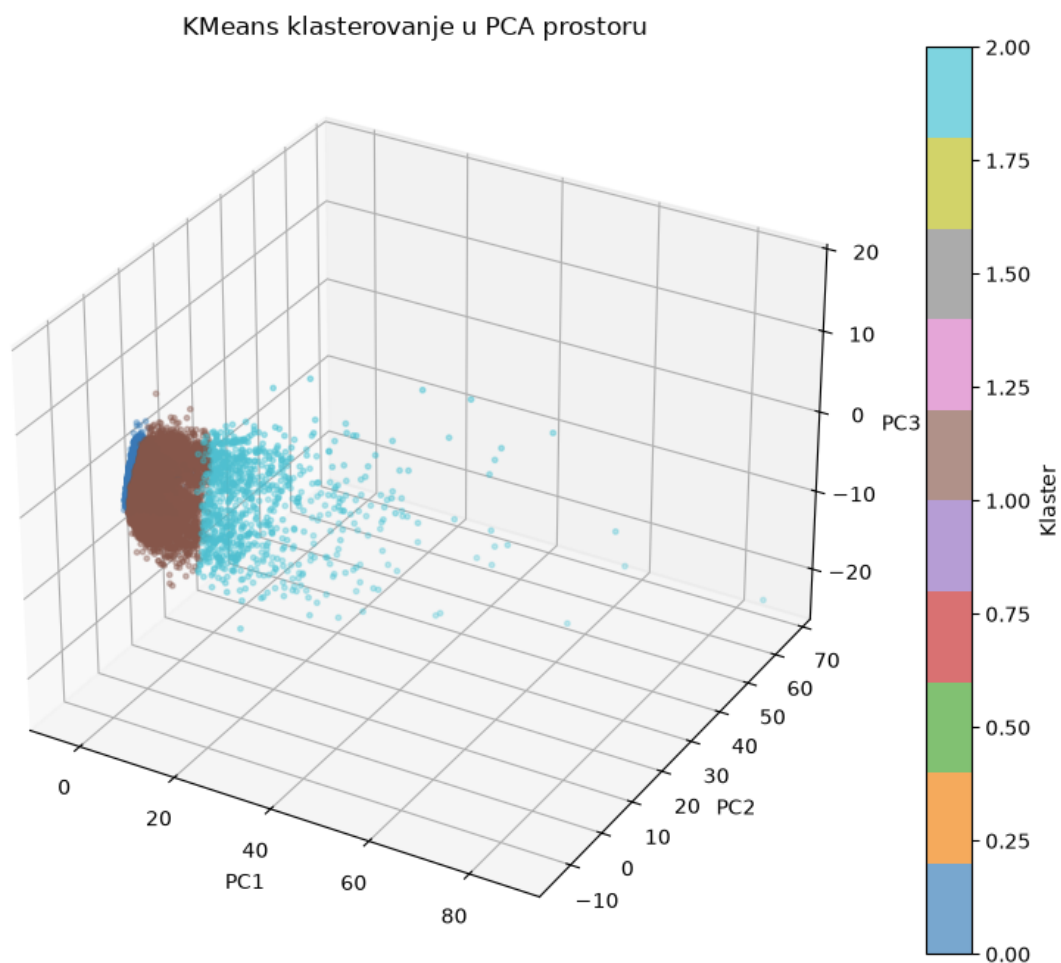
```

1 from mpl_toolkits.mplot3d import Axes3D
2
3 fig = plt.figure(figsize=(10,8))
4
5 ax = fig.add_subplot(
6     111,
7     projection="3d"
8 )
9
10 scatter = ax.scatter(
11     X_pca95["PC1"],

```

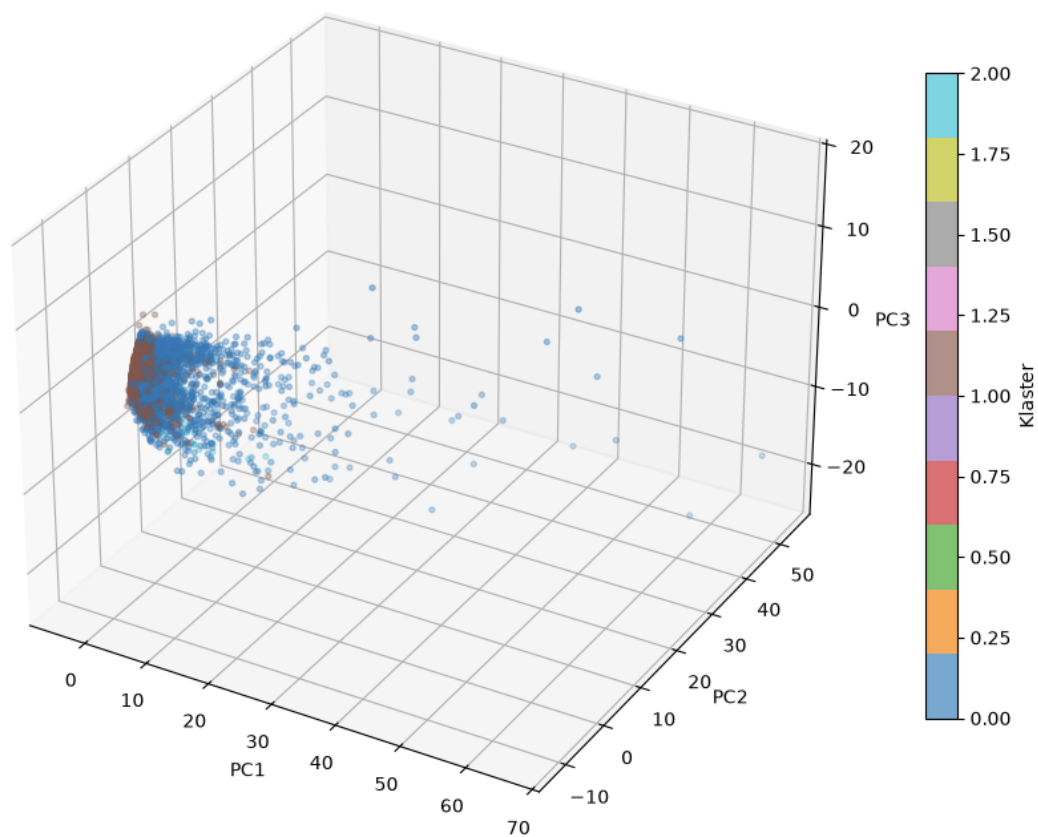
```
12     X_pca95["PC2"],
13     X_pca95["PC3"],
14     c=kmeans_labels,
15     cmap="tab10",
16     s=6,
17     alpha=0.65
18 )
19
20 ax.set_xlabel("PC1")
21 ax.set_ylabel("PC2")
22 ax.set_zlabel("PC3")
23
24 ax.set_title("KMeans klasterovanje u PCA prostoru")
25
26 plt.colorbar(
27     scatter,
28     label="Klaster"
29 )
30
31 plt.show()
```

Listing 23: Trodimenzionalna PCA projekcija

16.4.1. X_no_countries_no_club

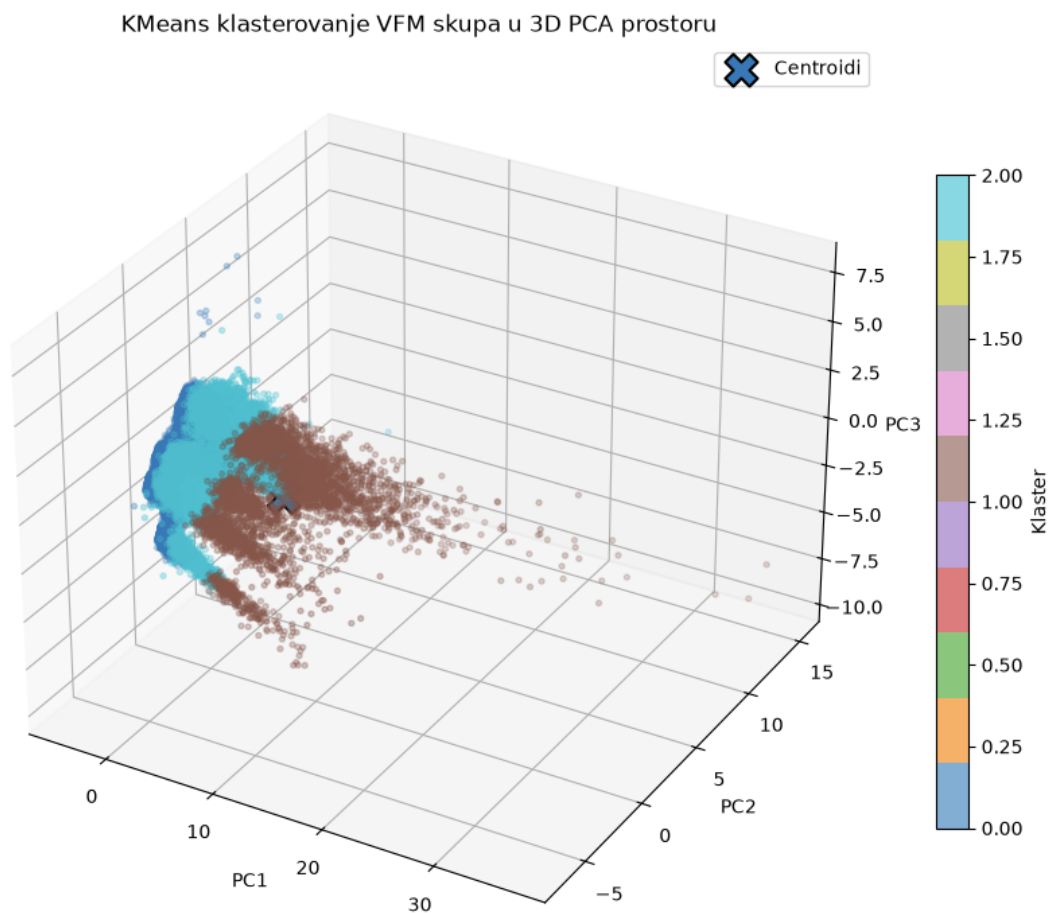
Slika 29: 3D vizualizacija klastera dobijenih KMeans algoritmom.

Hijerarhijsko klasterovanje u 3D PCA prostoru (stratifikovani uzorak od 10.000 igrača)



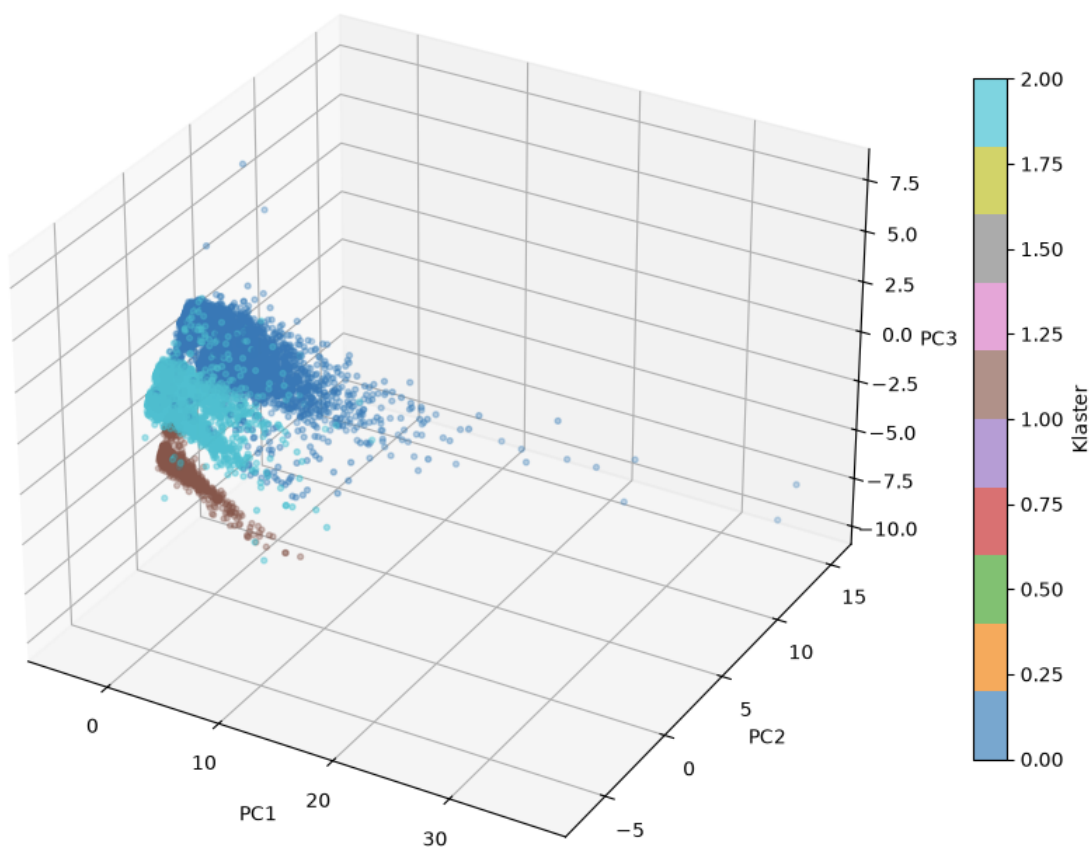
Slika 30: 3D vizualizacija klastera dobijenih hijerarhijskim algoritmom.

16.4.2. `x_value_for_money`



Slika 31: 3D vizualizacija klastera dobijenih KMeans algoritmom.

Hijerarhijsko klasterovanje VFM skupa u 3D PCA prostoru
(stratifikovani uzorak od 10.000 igrača)



Slika 32: 3D vizualizacija klastera dobijenih hijerarhijskim algoritmom.

17. Organizacija projekta po notebook-ovima

17.1. 01_preprocessing.ipynb

Ovaj notebook sadrži:

- učitavanje izvornih CSV datoteka;
- pregled dimenzija tabela;
- pregled tipova podataka;
- analizu nedostajućih vrednosti;
- uklanjanje stvarnih duplikata;
- izbor relevantnih osnovnih kolona;
- konverziju datuma;
- izračunavanje starosti;
- početnu obradu numeričkih i kategorijskih atributa;

- čuvanje pretprocesiranih tabela.

17.2. 02_feature_engineering.ipynb

Ovaj notebook sadrži:

- agregiranje podataka o nastupima;
- izračunavanje ukupne, prosečne i maksimalne minutaže;
- izračunavanje broja utakmica i golova;
- konstrukciju atributa startera, rezervi i kapitena;
- konstrukciju pozicionih brojeva i procenata;
- agregiranje istorijskih tržišnih vrednosti;
- računanje rasta i procentualnog rasta vrednosti;
- spajanje svih atributa na nivou igrača;
- obradu nedostajućih vrednosti nakon spajanja;

17.3. 03_feature_reduction.ipynb

Ovaj notebook sadrži:

- razdvajanje numeričkih i kategorijskih atributa;
- one-hot enkodiranje;
- standardizaciju;
- formiranje kompletnog skupa;
- formiranje skupa bez klupskih atributa;
- ograničavanje država na 30 najčešćih;
- PCA sa 90% objašnjene varijanse;
- formiranje skupa bez država
- formiranje skupa bez država i klubova
- PCA sa 95% objašnjene varijanse;
- čuvanje pripremljenih matrica.

17.4. 04_clustering.ipynb

Ovaj notebook sadrži:

- opštu funkciju za izvršavanje algoritama;
- K-sredina za više vrednosti broja klastera;
- GMM za više vrednosti broja komponenti;

- hijerarhijsko klasterovanje nad uzorkom;
- spektralno klasterovanje nad uzorkom;
- DBSCAN za više kombinacija parametara;
- računanje evaluacionih metrika;
- merenje vremena;
- čuvanje tabela rezultata.

17.5. 04_cluster_analysis.ipynb

Ovaj notebook sadrži:

- opštu funkciju za izučavanje klastera;
- izučavanje klastera dobijenih različitim algoritmima;
- prikaz rezultata;

17.6. 06_more_specific_clustering.ipynb

Ovaj notebook sadrži:

- izdvajanje novog skupa podataka za Value-for-money analizu
- klasterovanje novog skupa podataka različitim algoritmima
- analiziranje dobijenih klastera

17.7. 07_cluster_visualization.ipynb

Ovaj notebook sadrži:

- učitavanje sačuvanih oznaka klastera;
- 2D PCA vizualizacije;
- 3D PCA vizualizacije;

18. Čuvanje rezultata

Zbog dugog vremena izvršavanja rezultati su čuvani na disku.

Za svaki eksperiment čuvani su:

- naziv skupa;
- naziv algoritma;
- parametri;
- broj klastera;

- šum;
- silhouette koeficijent;
- Davies–Bouldin koeficijent;
- Calinski–Harabasz koeficijent;
- vreme izvršavanja;

Predložena struktura direktorijuma je:

```
project/  
|  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   `-- clusters/  
|  
|-- notebooks/  
|   |-- 01_preprocessing.ipynb  
|   |-- 02_feature_engineering.ipynb  
|   |-- 03_feature_reduction.ipynb  
|   |-- 04_clustering.ipynb  
|   |-- 05_cluster_analysis.ipynb  
|   |-- 06_more_specific_clustering.ipynb  
|   `-- 07_cluster_visualization.ipynb  
|  
|-- results/  
|   |-- metrics/  
|   |-- labels/  
|   `-- figures/  
|  
|-- requirements.txt  
`-- README.md
```

```
1 labels_df = pd.DataFrame({  
2     "player_id": player_ids,  
3     "cluster": labels  
4 })  
5  
6 labels_df.to_csv(  
7     "../data/processed/clusters/"  
8     "kmeans_x_full_k5.csv",  
9     index=False
```

10

)

Listing 24: Čuvanje oznaka klastera

```
1 results_df.to_csv(  
2     "../results/metrics/clustering_results.csv",  
3     index=False  
4 )
```

Listing 25: Čuvanje tabele rezultata

19. Računarski izazovi

19.1. Veliki broj redova

Tabele nastupa, postava i događaja sadrže veliki broj redova.

Zbog toga su primenjivani:

- izbor samo potrebnih kolona;
- filtriranje pre spajanja;
- agregiranje pre spajanja;
- brisanje privremenih objekata;
- čuvanje međurezultata;
- ponovno učitavanje samo gotovih matrica.

19.2. Veliki broj atributa

One-hot enkodiranje država, državljanstava, pozicija i klupskih informacija dovelo je do velikog broja dimenzija.

Problem je ublažen:

- grupisanjem retkih država;
- uklanjanjem klupskih atributa u jednoj verziji;
- PCA redukcijom;
- posebnim VFM skupom.

19.3. Memorijska zahtevnost hijerarhijskog algoritma

Pokušaj izvršavanja Ward-ovog hijerarhijskog klasterovanja nad kompletnim skupom doveo je do prekida rada kernela.

Rešenje je bilo korišćenje stratifikovanog uzorka od 10 000 igrača.

19.4. Zahtevnost spektralnog klasterovanja

Spektralno klasterovanje zahteva konstrukciju grafa sličnosti i sopstvenu dekompoziciju Laplasijana.

Zbog toga je takođe ograničeno na 10 000 instanci i korišćena je sličnost zasnovana na najbližim susedima.

19.5. Dugo izvršavanje DBSCAN algoritma

Pojedine konfiguracije DBSCAN algoritma zahtevale su više od 20 minuta i više gigabajta memorije.

Zbog toga rezultati nisu ponovo računati kada su već bili sačuvani. Dalji eksperimenti usmereni su samo na nedostajuće konfiguracije.

19.6. Cena računanja silhouette koeficijenta

Silhouette koeficijent zahteva računanje velikog broja rastojanja. Za velike skupove može predstavljati značajan deo ukupnog vremena.

Kod naročito velikih eksperimenata moguće je koristiti uzorak:

```
1 silhouette = silhouette_score(  
2     X,  
3     labels,  
4     sample_size=10_000,  
5     random_state=42  
6 )
```

Listing 26: Silhouette koeficijent nad uzorkom

Međutim, kada je bilo računarski izvodljivo, korišćen je ceo skup dodeljenih instanci.

20. Diskusija

20.1. Uticaj izbora atributa

Rezultati su pokazali da izbor atributa snažno utiče na dobijene klastere.

Kada su uključeni atributi države i kluba, klasteri mogu više odražavati klupski ili geografski kontekst.

Kada se klupski atributi uklone, veći značaj dobijaju individualne karakteristike igrača.

VFM skup dodatno usmerava algoritam ka:

- tržišnoj vrednosti;
- učinku;
- minutaži;
- statusu igrača u timu.

20.2. Uticaj PCA transformacije

PCA smanjuje broj dimenzija i uklanja deo redundantnosti između korelisanih atributa.

Prednosti PCA verzija bile su:

- kraće vreme izvršavanja;
- manja potrošnja memorije;
- ublažavanje problema visoke dimenzionalnosti;
- stabilnije računanje rastojanja.

Glavni nedostatak je smanjena neposredna interpretabilnost, jer glavna komponenta predstavlja linearnu kombinaciju velikog broja originalnih atributa.

Zato su PCA rezultati dopunjeni analizom najznačajnijih originalnih atributa.

20.3. Poređenje algoritama

K-sredina je bila najpraktičnija za brzo poređenje većeg broja verzija skupa.

GMM je omogućio fleksibilniji probabilistički model i meke pripadnosti, ali nije se pokazao kao najbolji na ovom skupu podataka.

Hijerarhijsko klasterovanje pružilo je uvid u strukturu spajanja grupa, ali nije bilo primenjivo na ceo skup.

Spektralno klasterovanje bilo je korisno za ispitivanje nelinearne strukture, ali uz veliku računarsku cenu.

DBSCAN je pokazao da skup može sadržati veoma guste jezgrene grupe, veliki broj manjih grupa i dosta šuma. Njegovi rezultati su, međutim, bili veoma osetljivi na parametre.

20.4. Problem neuravnoteženih klastera

Neke konfiguracije dale su visok silhouette rezultat, ali vrlo neuravnotežene klasterne.

To može nastati kada:

- jedan klaster sadrži većinu prosečnih igrača;
- mali broj ekstremnih igrača formira udaljene klastere;
- golmani budu veoma jasno odvojeni od ostalih;
- tržišna vrednost dominira nad drugim atributima.

20.5. Interpretabilnost kao kriterijum

Klasterovanje nije korisno samo ako daje dobru matematičku podelu. Potrebno je da grupe imaju i razumljivo značenje.

Dobar rezultat treba da omogući opis poput:

Klaster sadrži mlade igrače sa malom minutažom, niskom trenutnom vrednošću, ali pozitivnim trendom tržišne vrednosti.

Ako se klaster ne može razlikovati od drugih na osnovu prosečnih karakteristika, njegov praktični značaj je ograničen čak i kada su metrike prihvatljive.

21. Ograničenja projekta

21.1. Kvalitet izvornih podataka

Rezultati zavise od potpunosti i tačnosti izvornih podataka.

Za pojedine igrače nedostajali su:

- datum rođenja;
- visina;
- dominantna noga;
- država rođenja;
- broj reprezentativnih nastupa;
- istorija tržišne vrednosti;
- detaljna statistika nastupa.

Imputacija omogućava primenu algoritama, ali ne može nadoknaditi stvarno nepoznatu informaciju.

21.2. Tržišna vrednost nije čista mera kvaliteta

Tržišna vrednost zavisi od brojnih faktora:

- kvaliteta igrača;
- starosti;

- dužine ugovora;
- lige;
- kluba;
- medijske izloženosti;
- očekivanog budućeg razvoja;
- opšteg stanja tržišta.

Zbog toga se ne može tumačiti kao direktna mera sportskog kvaliteta.

21.3. Različiti periodi karijere

Agregiranje svih nastupa može objediniti više faza karijere jednog igrača.

Igrač koji je ranije bio standardni prvotimac, ali trenutno ima malu minutažu, može imati profil koji predstavlja kombinaciju prošlog i trenutnog stanja.

21.4. PCA i gubitak interpretabilnosti

Iako PCA čuva veliki deo varijanse, deo informacija se odbacuje. Takođe, glavne komponente nisu direktno sportski atributi.

21.5. Uzorci kod skupih algoritama

Hijerarhijsko i spektralno klasterovanje nisu primenjeni nad svim igračima.

Stratifikovani uzorak čuva raspodelu glavnih pozicija, ali ne garantuje da će sačuvati svaku malu i retku podgrupu.

22. Zaključak

U projektu je razvijen kompletan proces klasterovanja profesionalnih fudbalera, počevši od više sirovih i međusobno povezanih tabela, pa sve do interpretacije i vizualizacije dobijenih grupa.

Najveći deo rada odnosio se na pripremu podataka. Izvorne tabele su agregirane na nivou igrača, nakon čega su formirani atributi koji opisuju:

- minutažu;
- broj golova;
- ulogu startera, rezerve i kapitena;
- pozicionu fleksibilnost;
- trenutnu i istorijsku tržišnu vrednost;

- osnovne demografske i fudbalske karakteristike.

Formirano je više verzija skupa kako bi se odvojeno ispitao uticaj klupskih, geografskih, pozicionih, vrednosnih i konkretno igračkih atributa.

PCA je omogućio značajno smanjenje broja dimenzija uz zadržavanje 90–95% ukupne varijanse. Time su smanjeni računarski zahtevi i ublaženi problemi koji nastaju pri računanju rastojanja u veoma visokodimenzionalnom prostoru.

Primena više algoritama pokazala je da ne postoji jedno univerzalno najbolje klasterovanje.

K-sredina je bio su pogodan za sistematsko poređenje većeg broja konfiguracija. Hijerarhijsko i spektralno klasterovanje pružili su dodatni uvid u strukturu podataka, ali su zbog računarske složenosti primenjeni na uzorku. DBSCAN je pokazao mogućnost otkrivanja gustih grupa i šumova, uz izraženu osetljivost na izbor parametara.

Konačna procena rezultata zato nije zasnovana samo na predloženim metrikama. U obzir su uzeti veličina i stabilnost klastera, raspodela pozicija, prosečne karakteristike igrača, vreme izvršavanja i mogućnost sportske interpretacije.

Projekat pokazuje da klasterovanje može biti koristan alat za otkrivanje profila igrača, ali samo kada je praćeno pažljivom pripremom podataka, poređenjem više skupova atributa i detaljnom interpretacijom dobijenih grupa.

23. Mogući pravci daljeg rada

Projekat se može proširiti na više načina:

- razdvajanje igrača prema periodima karijere;
- klasterovanje unutar pojedinačnih pozicija;
- uključivanje atributa asistencija i naprednih statistika;
- davanje većeg značaja novijim utakmicama;
- poređenje klastera između sezona;
- izgradnja sistema za pronalaženje sličnih igrača;
- predviđanje prelaska igrača iz jednog profila u drugi;
- uključivanje poziciono specifičnih atributa.

Posebno koristan nastavak bio bi razvoj sistema koji za izabranog igrača pronalazi najbližije igrače prema standardizovanom prostoru atributa. Takav sistem mogao bi imati primenu u skautingu i analizi potencijalnih transfera.

A. Pokretanje projekta

A.1. Kreiranje virtuelnog okruženja

Na Linux ili WSL sistemu:

```
1 python3 -m venv venv-ip2
2 source venv-ip2/bin/activate
```

Listing 27: Kreiranje virtuelnog okruženja

Na Windows sistemu:

```
1 python -m venv venv-ip2
2 venv-ip2\Scripts\activate
```

Listing 28: Aktiviranje okruženja na Windows-u

A.2. Instalacija zavisnosti

```
1 python -m pip install --upgrade pip
2 pip install -r requirements.txt
```

Listing 29: Instalacija paketa

Primer sadržaja datoteke `requirements.txt`:

```
numpy
pandas
scipy
scikit-learn
matplotlib
jupyter
notebook
psutil
joblib
```

A.3. Pokretanje Jupyter Notebook-a

```
1 jupyter notebook
```

Listing 30: Pokretanje Jupyter okruženja

Notebook-ove treba pokretati sledećim redosledom:

1. `01_preprocessing.ipynb`

2. 02_feature_engineering.ipynb
3. 03_feature_reduction.ipynb
4. 04_clustering.ipynb
5. 05_cluster_analysis.ipynb
6. 06_more_specific_clustering.ipynb
7. 07_cluster_visualization.ipynb

Ako su obrađeni skupovi i rezultati klasterovanja već sačuvani, nije potrebno ponovo pokretati sve prethodne notebook-ove.

B. Korišćene biblioteke

Tabela 2: Glavne Python biblioteke

Biblioteka	Namena
pandas	Učitavanje, filtriranje, agregiranje, spajanje i čuvanje tabelarnih podataka.
numpy	Numeričke operacije i rad sa matricama.
scikit-learn	Standardizacija, PCA, algoritmi klasterovanja i evaluacione metrike.
scipy	Hijerarhijsko povezivanje i dendrogrami.
matplotlib	Dvodimenzionalne i trodimenzionalne vizualizacije.